Nesne Yönelimli Programlama

Vize Konu Anlatımı – Bütün Konular

Chapter 1, 2, 3, 7, 11, 13 ve UML sunumu; ISO/IEC 25010

9 Kasım 2026

İçindekiler

1 Yazılım Mühendisliğinin Kapsamı (Chapter 1) 2

1.1 Tarihsel boyut: yazılım krizi (Historical aspects) . . . 2
1.2 Ekonomik boyut (Economic aspects) . . . 2
1.3 Bakım boyutu (Maintenance aspects) . . . 2
1.3.1 Yaşam döngüsü modeli ve klasik fazlar . . . 3
1.3.2 Üç bakım türü . . . 3
1.3.3 Klasik ve modern bakım tanımları . . . 3
1.3.4 Bakımın önemi . . . 4
1.4 Gereksinim, analiz ve tasarım boyutu . . . 4
1.5 Takım çalışması boyutu . . . 4
1.6 Neden ayrı bir planlama, test ve dokümantasyon fazı yok? . . . 5
1.7 Nesne yönelimli paradigma . . . 5
1.7.1 Yapısal paradigma neden yetersiz kaldı? . . . 5
1.7.2 Nesne . . . 5
1.7.3 Nesne yönelimli paradigmanın güçlü yanları . . . 5
1.7.4 Responsibility-driven design (design by contract) . . . 6
1.7.5 Analiz/tasarım "tümseği" (hump) . . . 6
1.7.6 Perspektif . . . 6
1.8 Terminoloji . . . 6
1.9 Etik konular (Ethical issues) . . . 7
1.10 Kısa tanım soruları ("What is it?") . . . 7

2 Yaşam Döngüsü Modelleri (Chapter 2) 9

2.1 Teoride ve pratikte yazılım geliştirme . . . 9
2.2 Winburg mini vaka çalışması . . . 9
2.2.1 Evolution-tree (evrim ağacı) modeli . . . 9
2.2.2 Winburg'un dersleri . . . 10

2.3 Teal Tractors mini vaka çalışması ve moving target problem . . . 10
2.4 Yineleme ve artırım (Iteration and Incrementation) . . . 10
2.4.1 Miller yasası ve stepwise refinement . . . 10
2.4.2 Iteration ve incrementation . . . 11
2.4.3 Klasik fazlar ve iş akışları (workflows) . . . 11
2.4.4 Winburg yeniden: artırımlar ve bakım . . . 11
2.5 Yinelemeli-artırımlı modelin riskleri ve güçlü yanları . . . 11
2.6 Diğer yaşam döngüsü modelleri (kısaca) . . . 12
2.7 Stable Intermediate Forms (kararlı ara formlar) . . . 12
2.8 Kısa tanım soruları . . . 13

3 Unified Process (Chapter 3) 15
3.1 Unified Process nedir? . . . 15
3.2 Beş çekirdek iş akışı (core workflows) . . . 15
3.2.1 Requirements workflow . . . 15
3.2.2 Analysis workflow . . . 16
3.2.3 Design workflow . . . 16
3.2.4 Implementation workflow . . . 16
3.2.5 Test workflow . . . 16
3.2.6 Postdelivery maintenance ve retirement . . . 17
3.3 Unified Process'in dört fazı . . . 17
3.3.1 Inception phase (başlangıç fazı) . . . 17
3.3.2 Elaboration phase (ayrıntılandırma fazı) . . . 18
3.3.3 Construction phase (yapım fazı) . . . 18
3.3.4 Transition phase (geçiş fazı) . . . 18
3.4 Tek ve iki boyutlu yaşam döngüsü modelleri . . . 19
3.5 IEEE Software Project Management Plan (SPMP) . . . 19
3.6 Yazılım sürecini iyileştirme (Improving the software process) . . . 20
3.7 Kısa tanım soruları . . . 21

4 Modüllerden Nesnelere: Cohesion ve Coupling (Chapter 7) 22
4.1 Neden modülerleştirme? . . . 22
4.2 Modül nedir? . . . 22
4.2.1 Bilgisayar tasarımı örneği . . . 22
4.2.2 Bir modülün function, logic ve context'i . . . 23
4.3 Cohesion (modül içi uyum): 7 seviye . . . 23
4.3.1 Cohesion türünü bulma algoritması (sınavda koddan tür bulurken) . . . 26
4.3.2 Hocanın derste çözdüğü örnekler . . . 26
4.4 Coupling (modüller arası bağlılık): 5 seviye . . . 27
4.4.1 Coupling türünü bulma algoritması . . . 29

4.5 Sınav tipi örnek: kodu verip tür ve düzeltme isteme . . . 29
4.6 Data encapsulation, ADT ve information hiding . . . 30
4.6.1 İş kuyruğu örneği . . . 30
4.6.2 Abstract data type (ADT) . . . 30
4.6.3 Information hiding . . . 31
4.6.4 Nesneler, kalıtım, polymorphism ve dynamic binding . . . 31
4.6.5 Nesne yönelimli paradigmanın güçlü ve zayıf yanları . . . 31
4.7 SOLID ilkeleri (giriş) . . . 31
4.8 Kısa tanım soruları . . . 32

5 ISO/IEC 25010 Yazılım Ürün Kalitesi Modeli 33
5.1 Model nedir? . . . 33
5.2 Hocanın tablosu: 9 karakteristik, 40 alt karakteristik (ISO/IEC 25010:2023) . . . 33
5.3 Alt karakteristiklerin anlamları . . . 34
5.4 2011 sürümü (8 karakteristik, 31 alt karakteristik) . . . 35
5.5 Sınavda cevap şablonu . . . 35

6 Gereksinim İş Akışı ve Use Case Diyagramları (Chapter 11) 36
6.1 Gereksinim iş akışının amacı . . . 36
6.2 Gereksinim iş akışının adımları . . . 36
6.2.1 Alanı anlamak ve sözlük (glossary) . . . 36
6.2.2 İş modeli ve bilgi toplama teknikleri . . . 36
6.3 Use case ve aktör . . . 37
6.4 Use case diyagramının notasyonu . . . 37
6.5 Use case açıklaması (description) . . . 38
6.6 Hocanın çözdüğü örnek 1: Adres defteri . . . 39
6.7 Hocanın çözdüğü örnek 2: E-ticaret sitesi (2024–25 vize sorusu) . . . 40
6.8 İlk gereksinimler: functional ve nonfunctional . . . 41
6.9 MSG Foundation vaka çalışması . . . 41
6.9.1 Alan bilgisi . . . 41
6.9.2 İlk iş modeli . . . 41
6.9.3 Haftalık fon formülü . . . 41
6.9.4 Yinelemelerle gereksinimlerin revizyonu . . . 42
6.10 Diğer konular . . . 42
6.11 Kısa tanım soruları . . . 43

7 Nesne Yönelimli Analiz (Chapter 13) 44
7.1 Analiz iş akışı ve üç sınıf türü . . . 44
7.2 Entity sınıflarını çıkarmanın üç adımı . . . 44
7.3 Asansör problemi (elevator problem) . . . 44

7.3.1 Functional modeling . . . 45
7.3.2 Entity class modeling: isim çıkarma (noun extraction) . . . 45
7.3.3 CRC kartları . . . 45
7.3.4 Dynamic modeling: statechart . . . 46
7.3.5 Test workflow: CRC ile hataların bulunması . . . 46
7.3.6 God class ve dağıtık mimari . . . 46
7.4 Boundary ve control sınıflarını çıkarmak . . . 46
7.5 MSG Foundation vaka çalışması . . . 46
7.5.1 İlk sınıf diyagramı (noun extraction) . . . 46
7.5.2 İlk dinamik model . . . 47
7.5.3 Boundary ve control sınıfları . . . 47
7.6 Use-case realization . . . 47
7.6.1 Collaboration diagram . . . 48
7.6.2 Eşdeğer sequence diagram . . . 49
7.6.3 Sequence mı, collaboration mı? . . . 49
7.7 Hocanın tam çözümlü örneği: Adres Defteri . . . 50
7.7.1 Sınıf diyagramı . . . 50
7.7.2 CRC kartları (hocanın kâğıdından) . . . 51
7.7.3 Kişi Ekle: collaboration ve sequence diyagramı . . . 51
7.8 Diğer konular . . . 52
7.9 Kısa tanım soruları . . . 52

8 UML: Sınıf İlişkileri, Sequence ve Diğer Diyagramlar 53
8.1 UML nedir, neden kullanılır? . . . 53
8.2 Class diagram . . . 53
8.2.1 Sınıf notasyonu . . . 54
8.2.2 Sınıflar arası ilişkiler . . . 54
8.2.3 İlişkilerin C# karşılıkları . . . 55
8.2.4 Çokluklar (multiplicities) . . . 56
8.3 UML sunumu sayfa 35: sınıf diyagramını yorumlama . . . 56
8.4 Sequence diagram notasyonu . . . 57
8.5 Diğer UML diyagramları . . . 58
8.6 Kısa tanım soruları . . . 58

1 Yazılım Mühendisliğinin Kapsamı (Chapter 1)

Bu bölümde ne var, sınavda nasıl çıkıyor?

Chapter 1 kitabın giriş bölümüdür: *neden yazılım mühendisliğine ihtiyaç var, bakım (maintenance) neden bu kadar önemli, neden ayrı bir planlama / test / dokümantasyon fazı yok* ve *nesne yönelimli paradigma neden daha iyi* sorularının cevabıdır. Sınavda bu bölümden genellikle

- kısa tanım soruları ("What is regression fault / moving target / modern maintenance?"),
- test (çoktan seçmeli) soruları (bakım türleri, ayrı faz olmayan etkinlik, modern bakım tanımı) ve
- **etik kurallar** (Software Engineering Code of Ethics, 8 ilke) gelir.

Hoca dersi İngilizce anlatır; cevaplarda terimlerin **İngilizcesini** kullanın. Bu dosyada her başlığın sonunda sınava yazılabilecek kısa İngilizce cevap kalıpları verilmiştir.

1.1 Tarihsel boyut: yazılım krizi (Historical aspects)

- **1968 NATO Konferansı (Garmisch, Almanya):** "software engineering" terimi burada ortaya atıldı. Amaç **software crisis**'i (yazılım krizi) çözmekti.
- Yazılım krizinin üç belirtisi: yazılım **geç teslim edilir (late)**, **bütçeyi aşar (over budget)** ve **içinde kalıntı hatalar vardır (with residual faults)**.
- **Standish Group 2006 verisi:** projelerin yalnızca %35'i başarılı; %46'sı tamamlandı ama geç / bütçe üstü / eksik özellikli (challenged); %19'u iptal edildi ya da hiç kullanılmadı. Yani "üç projeden ancak biri başarılı".
- **Cutter Consortium 2002:** kurumların %78'i dava ile biten anlaşmazlık yaşamış. Bunların %67'sinde teslim edilen sistem söz verilen işlevi karşılamamış, %56'sında teslim tarihi defalarca kaymış, %45'inde hatalar sistemi kullanılamaz hâle getirmiş.
- Sonuç: kriz çözülmedi; kitap buna **software depression** (uzun süren, kötü seyirli) der.

Sınav cevabı kalıbı

Software crisis: software is delivered late, over budget and with residual faults. The term software engineering was coined at the 1968 NATO conference in Garmisch to solve this crisis.

1.2 Ekonomik boyut (Economic aspects)

Örnek: yeni kodlama yöntemi CM_{new}, eskisi CM_{old}'dan %10 daha hızlı. Kullanmalı mıyız?

- **Sağduyu cevabı:** Tabii ki!
- **Yazılım mühendisliği cevabı:** önce (1) personelin **eğitim maliyeti**, (2) yeni teknolojiye geçişin **etkisi** (yeni araçlar, ek personel, düzenin bozulması) ve (3) yöntemin **bakıma etkisi** düşünülmelidir.

Kodlama + birim testi toplam maliyetin yalnızca yaklaşık $0{,}34 \times 0{,}25 = \%8{,}5$'idir. Bu yüzden kodlamayı %10 hızlandırmak toplam maliyeti en fazla %0,85 düşürür. Buna karşılık bakım maliyetini %10 düşüren bir teknik toplam maliyeti $0{,}10 \times \%75 = \%7{,}5$ düşürür. **Asıl kazanç bakımdadır.**

1.3 Bakım boyutu (Maintenance aspects)

1.3.1 Yaşam döngüsü modeli ve klasik fazlar

- **Life-cycle model:** yazılım geliştirirken izlenecek adımların (fazların) teorik tarifi. "Ne yapılmalı".
- **Life cycle:** belirli bir ürün için *gerçekte* uygulanan adımlar.

Klasik (waterfall, 1970) modelin fazları ve her fazdaki etkinlikler:

Faz	Yapılanlar
1. Requirements	Kavramı keşfet (explore the concept), müşterinin gereksinimlerini ortaya çıkar (elicit).
2. Analysis (specification)	Gereksinimleri analiz et, **specification document** hazırla, **SPMP** (software project management plan) hazırla. "Ürün *ne* yapacak?"
3. Design	Mimari tasarım (architectural design: modüllere ayırma), ardından ayrıntılı tasarım (detailed design: her modülün veri yapısı ve algoritması). "Ürün bunu *nasıl* yapacak?"
4. Implementation	Kodlama, birim testi (unit testing), bütünleştirme (integration), kabul testi (acceptance testing).
5. Postdelivery maintenance	Corrective, perfective ve adaptive bakım.
6. Retirement	Ürünün işlevine artık ihtiyaç kalmayınca kullanımdan kaldırılması.

1.3.2 Üç bakım türü

Tür	Tanım	Örnek
Corrective (düzeltici)	Kalıntı hataların giderilmesi; spesifikasyon değişmez (*software repair*).	Faiz yanlış hesaplanıyor, düzeltilir.
Perfective (mükemmelleştirici)	Müşterinin ürünün etkinliğini artıracağını düşündüğü değişiklikler: yeni işlev, daha kısa yanıt süresi.	Rapora yeni sütun eklemek, sistemi hızlandırmak.
Adaptive (uyarlayıcı)	Ürünün çalıştığı **ortam** değişince yapılan değişiklikler: yeni donanım / işletim sistemi, yeni yasal düzenleme.	KDV oranı değişti; Teal Tractors'ta Kanada vergisi ve CAD para birimi.

Perfective ve adaptive birlikte **enhancement** (software update) diye anılır. "Reliable" bir bakım türü değildir (çıkmış test sorusu).

1.3.3 Klasik ve modern bakım tanımları

Klasik tanım (development-then-maintenance model): Ürün kurulduktan sonra yapılan her değişiklik bakımdır. Bu **temporal** (zamana bağlı) bir tanımdır: bir etkinliğin bakım mı geliştirme mi olduğu *ne zaman yapıldığına* bağlıdır.

- *Sonuç 1:* Bir hata kurulumdan **bir gün sonra** bulunup düzeltilirse klasik bakım; aynı hata kurulumdan **bir gün önce** düzeltilirse klasik geliştirme sayılır.
- *Sonuç 2:* Kurulumdan sonra müşteri işlev isterse perfective maintenance; aynı değişikliği kurulumdan hemen önce isterse (**moving target problem**) klasik geliştirme sayılır.
- Ayrıca günümüzde sıfırdan geliştirme nadirdir, **yeniden kullanım (reuse)** yaygındır.

Modern tanım (ISO/IEC 12207, 1995; IEEE/EIA de kabul etti): Bakım, *bir yazılım ürünü bir problem nedeniyle ya da iyileştirme / uyarlama ihtiyacıyla değiştirildiğinde* gerçekleşen süreçtir.

Yani **yazılım ne zaman değiştirilirse bakım vardır**; kurulumdan önce ya da sonra olması fark etmez. Bu **operational** (işleme dayalı) bir tanımdır.

- **Postdelivery maintenance:** teslim ve kurulumdan sonraki değişiklikler (IEEE 1990).
- **Modern maintenance** (ya da sadece maintenance): *her zaman* yapılan corrective, perfective veya adaptive bakım.

Sınav cevabı kalıbı

Classical maintenance is a temporal definition: any change after installation is maintenance (development-then-maintenance model). Modern maintenance (ISO/IEC 1995) is an operational definition: maintenance occurs whenever software is modified because of a problem or a need for improvement or adaptation, regardless of whether it takes place before or after installation.

1.3.4 Bakımın önemi

- Kötü yazılım atılır; **iyi yazılım 10–20 yıl bakımla yaşar**. Yazılım sürekli değişen gerçek dünyanın bir modelidir.
- 1976–1981 arası maliyetin yaklaşık **%67**'si, 1992–1998 arası **%75**'i (birçok kurumda %70–80) postdelivery maintenance'a gider.
- Geliştirme fazlarının kendi içindeki oranları ise neredeyse hiç değişmedi (gereksinim+analiz %18–21, tasarım %18–19, kodlama+birim testi %34–36, bütünleştirme %24–29).

1.4 Gereksinim, analiz ve tasarım boyutu

- **Bir hata ne kadar erken bulunursa o kadar ucuza düzeltilir.** Gereksinim fazındaki bir hatayı bakım aşamasında düzeltmek yaklaşık 100–200 kat pahalıdır (Boehm, logaritmik grafik).
- Erken düzeltmede genellikle sadece bir **doküman** değişir. Geç düzeltmede: kod ve doküman değişir, değişikliğin kendisi test edilir, **regression testing** yapılır, ürün müşterinin bilgisayar(lar)ına yeniden kurulur.
- Büyük projelerde hataların **%60–70**'i gereksinim, analiz ve tasarım hatasıdır. Jet Propulsion Laboratory incelemesi: spesifikasyonda sayfa başına 1,9; tasarımda 0,9; kodda 0,3 hata.
- Sonuç: gereksinim, analiz ve tasarım tekniklerini iyileştirmek hayati önemdedir.

1.5 Takım çalışması boyutu

Donanım ucuz olduğu için tek kişinin zamanında yazamayacağı kadar büyük ürünler yapılır; yazılımı **takımlar** geliştirir. Bunun iki sorunu vardır: modüller arası **arayüz (interfacing) problemleri** ve takım üyeleri arası **iletişim problemleri**.

1.6 Neden ayrı bir planlama, test ve dokümantasyon fazı yok?

Etkinlik	Neden ayrı faz değil?
Planning	Projenin başında neyin yapılacağını tam bilmediğimiz için baştan plan yapılamaz. Klasik paradigmada başta gereksinim ve analiz fazlarının ön planı yapılır; SPMP ancak müşteri spesifikasyonları onayladıktan (*signed off*) sonra hazırlanır ve proje boyunca izlenir. **Planlama bütün yaşam döngüsü boyunca yapılır.**
Testing	Geliştirme bittikten sonra, teslimden önce test etmek çok geçtir. **Verification** her fazın sonunda yapılan testtir (geç), **validation** projenin sonunda yapılan testtir (çok geç). Test sürekli yapılmalıdır; sorumlusu her yazılımcı ve **SQA (software quality assurance) grubu**dur.
Documentation	Sonradan yazmak çok geçtir: kilit kişiler belgeleme bitmeden ayrılabilir; önceki fazın belgesi olmadan sonraki faz yapılamaz; belge olmadan test ve bakım yapılamaz. Dokümantasyon **diğer bütün etkinliklerle paralel** yürütülür ve her zaman güncel olmalıdır.

Çıkmış test sorusu

"Which one is NOT a separate phase during software development?" Seçenekler: implementation, analysis, design, **documentation**, requirements. Cevap: **documentation**. Aynı mantıkla *testing* ve *planning* de ayrı faz değildir.

Verification ve validation farkı (sık sorulur): *verification* "ürünü doğru mu yapıyoruz?" (her fazın sonunda, o fazın doğru tamamlandığının kontrolü); *validation* "doğru ürünü mü yaptık?" (projenin sonunda ürünün spesifikasyonu karşıladığının kontrolü).

1.7 Nesne yönelimli paradigma

1.7.1 Yapısal paradigma neden yetersiz kaldı?

Yapısal (structured, classical) paradigma başta başarılıydı ama **50.000 satırdan büyük ürünlerde** başarısız olmaya başladı; bakım toplam eforun %70–80'ine çıktı. Sebep: yapısal yöntemler ya **action oriented** (sonlu durum makineleri, veri akış diyagramları) ya da **data oriented** (varlık-ilişki diyagramları, Jackson yöntemi) idi, **ikisi birden değil**.

1.7.2 Nesne

Nesne yönelimli paradigmada **veri ve eylemler eşit önemdedir**. **Nesne (object):** hem veriyi hem de o veri üzerinde yapılan eylemleri içeren yazılım bileşenidir. *Örnek: banka hesabı.* Veri: hesap bakiyesi (`accountBalance`). Eylemler: para yatır (`deposit`), para çek (`withdraw`), bakiyeyi öğren (`determineBalance`). Klasik versiyonda bakiyenin nasıl tutulduğunu bütün modüller bilir; nesne yönelimli versiyonda bakiye nesnenin içinde gizlidir ve dışarıdan yalnızca bu üç metot ile erişilir (**information hiding**).

1.7.3 Nesne yönelimli paradigmanın güçlü yanları

1. **Information hiding** sayesinde bakım daha güvenlidir: **regression fault** (görünüşte ilgisiz bir yerde ortaya çıkan hata) ihtimali azalır.
2. Geliştirme kolaydır: nesnelerin genellikle fiziksel karşılıkları vardır, bu da modellemeyi kolaylaştırır.
3. İyi tasarlanmış nesneler **bağımsız birimlerdir**: gerçek dünyadaki varlıkla ilgili her şey o nesnededir (**encapsulation**); iletişim **mesaj göndererek** olur; bağımsızlığı **responsibility-driven design** artırır.

4. Klasik ürün kavramsal olarak tek birimdir; nesne yönelimli ürün bağımsız birimlerden oluştuğu için **karmaşıklığı azaltır**.
5. Nesneler bağımsız olduğundan **yeniden kullanımı (reuse)** destekler.

1.7.4 Responsibility-driven design (design by contract)

Hocanın derste sorduğu soru: "Başka şehirdeki annene çiçek göndermek istiyorsun, ne yaparsın?" Çiçek sitesini (kitapta 1-800-flowers, derste Çiçeksepeti) ararsın. Çiçekçi nerede, hangi şehirdeki çiçekçi teslim edecek, bilmezsin; bilmen de gerekmez. **Bir nesnenin metoduna, nesnenin iç yapısını bilmeden mesaj gönderirsin**; işi yapmak o nesnenin **sorumluluğudur**. Bu, information hiding'in bir uygulamasıdır.

1.7.5 Analiz/tasarım "tümseği" (hump)

- Yapısal paradigmada analiz (*ne*) ile tasarım (*nasıl*) arasında sert bir geçiş (*jolt*) vardır: modüller ancak mimari tasarımda ortaya çıkar.
- Nesne yönelimli paradigmada nesneler **object-oriented analysis** iş akışında ortaya çıkar; tasarımda bu nesneler tasarlanır, gerçekleştirmede kodlanır. Geçişler yumuşaktır, "tümsek" kalkar.

1.7.6 Perspektif

Nesne yönelimli paradigma doğru kullanılırsa klasik paradigmanın *bazı* (hepsi değil) sorunlarını çözer; kendi sorunları da vardır. Bugün eldeki en iyi alternatiftir ama gelecekte daha iyisiyle yer değiştirecektir.

1.8 Terminoloji

Terim	Anlamı
Client / developer / user	Yazılımı isteyen / yapan / kullanan.
Internal software	Müşteri ve geliştirici aynı kurumda.
Contract software	Müşteri ve geliştirici farklı kurumlarda.
COTS (commercial off-the-shelf)	Hazır paket, raftan satılan ticari yazılım.
Open-source software	Gönüllülerce geliştirilir ve bakılır. *Linus's Law:* "Yeterince göz varsa bütün hatalar sığdır."
Program / system / product	Kod parçası / birlikte çalışan programlar / bunların tamamı ile belgeleri.
Methodology (paradigm)	Yazılım üretme yolu (OO paradigm, classical paradigm).
Technique	Bir etkinliği yürütme yöntemi.
Mistake → fault → failure → error	Programcının yaptığı yanlışlık (**mistake**) koda bir hata (**fault**) olarak girer; yazılım çalışınca bu hata gözlenen yanlış davranışa (**failure**) yol açar; **error**, sonucun *ne kadar yanlış olduğunun ölçüsüdür* (IEEE 610.12; hoca: "a quantitative definition"). **Defect**: fault, failure veya error için genel (şemsiye) terim. "Bug" terimi sorumluluğu atar ("koda bir böcek girdi" yerine "hata yaptım" denmeli).

Nesne yönelimli terimler: nesnenin veri bileşeni *state variable / instance variable (Java) / field (C++) / attribute (genel)*; eylem bileşeni *member function (C++) / method (genel).*

1.9 Etik konular (Ethical issues)

! Hoca derste vurguladı [video 2.4]

"i ask this code of ethics in the exam okay please study this code of ethics … there are eight principles." 2025 WhatsApp grubunda da kitabın ilgili sayfasına "Bazen sınavlarda soruyor" notu düşülmüş.

Yazılım geliştiren ve bakımını yapan kişilerin özellikleri: **hard working, intelligent, sensible, up to date** ve her şeyden önce **ethical**. (WhatsApp'ta bunlara *lifelong learner* da eklenmiş; bu, 8. ilkeden gelir.)

ACM ve IEEE-CS ortak olarak **Software Engineering Code of Ethics and Professional Practice** (sürüm 5.2, 1999) yayımladı. Kısa sürümü bir giriş (preamble) ve **sekiz ilkeden** oluşur:

#	İlke	Açıklama (İngilizce aslı ve Türkçe özü)
1	**Public**	*Software engineers shall act consistently with the public interest.* Kamu yararına uygun davran.
2	**Client and Employer**	*…act in a manner that is in the best interests of their client and employer consistent with the public interest.* Kamu yararıyla çelişmeden müşteri ve işverenin çıkarını gözet.
3	**Product**	*…ensure that their products and related modifications meet the highest professional standards possible.* Ürün ve değişikliklerde en yüksek profesyonel standart.
4	**Judgment**	*…maintain integrity and independence in their professional judgment.* Mesleki yargıda dürüstlük ve bağımsızlık.
5	**Management**	*Software engineering managers and leaders shall subscribe to and promote an ethical approach to the management of software development and maintenance.* Yöneticiler etik yönetimi benimser ve teşvik eder.
6	**Profession**	*…advance the integrity and reputation of the profession consistent with the public interest.* Mesleğin itibarını ve dürüstlüğünü yükselt.
7	**Colleagues**	*…be fair to and supportive of their colleagues.* Meslektaşlara adil ve destekleyici ol.
8	**Self**	*…participate in lifelong learning regarding the practice of their profession and shall promote an ethical approach to the practice of the profession.* Ömür boyu öğren, etik yaklaşımı teşvik et.

Ezber ipucu

Sıra: **P**ublic, **C**lient and Employer, **P**roduct, **J**udgment, **M**anagement, **P**rofession, **C**olleagues, **S**elf. Dışarıdan içeriye doğru düşünün: toplum → müşteri → ürün → kendi yargın → yönetim → meslek → meslektaşlar → kendin.

1.10 Kısa tanım soruları ("What is it?")

Software crisis Software is delivered late, over budget and with residual faults.

Regression fault A fault in an apparently unrelated part of the software, caused by a change made elsewhere. Information hiding reduces regression faults.

Regression testing Re-running all previous test cases after a change to check that nothing else is broken.

Moving target problem A change in the requirements while the software is being developed (Chapter 2).

Verification / validation Verification: testing at the end of each phase. Validation: testing at the end of the project (whether the product satisfies the specification).

Modern maintenance Maintenance occurs whenever software is modified, before or after installation.

Object A software component that incorporates both data and the actions performed on that data.

Mistake / fault / failure / error A mistake is made by a human; its consequence is a fault in the code; a failure is the observed incorrect behavior caused by the fault; the error is the amount by which a result is incorrect (IEEE 610.12).

Information hiding Hiding the implementation details of a module so that other modules cannot depend on them; future changes are localized.

Encapsulation Gathering everything that relates to a real-world item (data and actions) into one unit.

Responsibility-driven design Sending a message to an object without knowing how it performs the task (design by contract).

2 Yaşam Döngüsü Modelleri (Chapter 2)

Bu bölümün sınavdaki yeri

Chapter 2'nin özü şudur: *gerçek hayatta yazılım doğrusal geliştirilmez; hatalar yapılır ve gereksinimler değişir. Bunun etkisini azaltmak için yinelemeli ve artırımlı (iterative and incremental) model ile sağlam (robust) bir mimari gerekir.* Son iki yılın vizelerinde bu bölümle ilişkili **Stable Intermediate Forms** sorusu (15 puan, açıklamalarıyla) geldi. Ayrıca *moving target problem*, *regression fault*, *Miller's law*, *iteration* ve *incrementation* kısa tanım olarak sorulur.

2.1 Teoride ve pratikte yazılım geliştirme

İdeal olarak yazılım Chapter 1'deki gibi **doğrusal (linear)** ve **sıfırdan (from scratch)** geliştirilir: requirements → analysis → design → implementation. Gerçek dünyada ise bu tamamen farklıdır, çünkü:

1. İnsanlar **hata yapar** (we make mistakes).
2. Ürün geliştirilirken **müşterinin gereksinimleri değişir** (moving target problem).

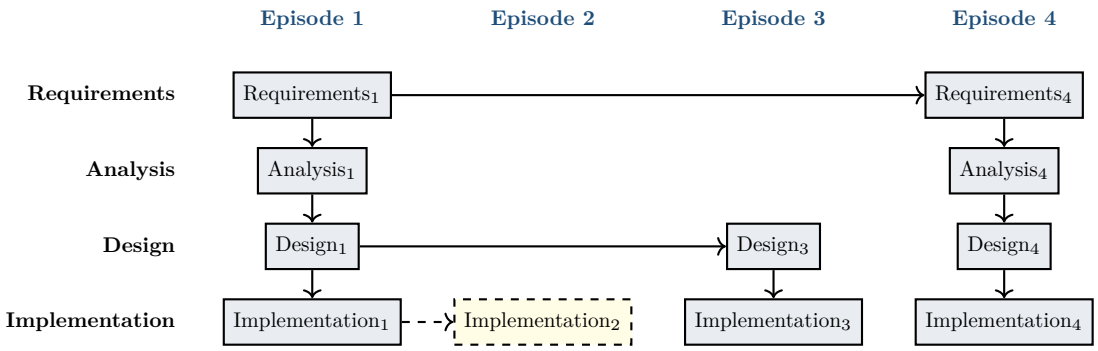
2.2 Winburg mini vaka çalışması

Winburg (Indiana) belediyesi trafiği azaltmak için otobüs sistemi kurar. Her otobüste yalnızca 1 dolarlık banknot kabul eden bir **ücret makinesi** vardır; yazılım bir görüntü tanıma algoritmasıyla banknotun geçerli olup olmadığına karar verir. Gereksinimler: ortalama yanıt süresi **1 saniyenin altında**, doğruluk **en az %98**.

Bölüm	Ne oldu?	Ne tür değişiklik?
Episode 1	Yazılımın ilk sürümü gerçekleştirildi.	–
Episode 2	Yanıt süresi 10 saniye çıktı. Sebep: %98 doğruluk için bütün hesaplarda *double precision* kullanılmış. Single precision'ın da yeterli olduğu anlaşıldı, kodda düzeltmeye başlandı.	**Implementation fault** → corrective
Episode 3	Değişiklik bitse bile süre 4,5 s olacaktı; sorun karmaşık algoritmaydı. Yeni, hızlı bir algoritmayla **yeniden tasarlandı** ve gerçekleştirildi.	**Design fault** → corrective
Episode 4	Belediye başkanı ürünü satmak için doğruluğun artırılmasını istedi. Yeni tasarımla doğruluk %99,5'e çıktı.	**Gereksinim değişikliği** → perfective
Epilogue	Birkaç yıl sonra sensörler eskidi, donanım değişti; yazılım yeniden yazıldı ve aynı sorunlar tekrar yaşandı.	Adaptive

2.2.1 Evolution-tree (evrim ağacı) modeli

Waterfall modeli geri besleme oklarıyla Winburg'u gösterebilir ama **olayların sırasını gösteremez**. Evolution-tree modeli her bölümde hangi artifact'ın hangi sürümünün üretildiğini ve olayların sırasını açıkça gösterir.



Kesikli kutu (Implementation_2) tamamlanmayan işi gösterir. Her bölümün sonunda bir **baseline** (o anki artifact'ların tam kümesi) vardır. Örnek: Episode 3 sonundaki baseline = $\textit{Requirements}_1$, $\textit{Analysis}_1$, $\textit{Design}_3$, $\textit{Implementation}_3$.

2.2.2 Winburg'un dersleri

Gerçek dünyada geliştirme Winburg'dan da kaotiktir. **Değişiklik her zaman gerekir**: yazılım sürekli değişen gerçek dünyanın modelidir ve yazılımcılar insandır, hata yapar.

2.3 Teal Tractors mini vaka çalışması ve moving target problem

Teal Tractors yazılımı yapılırken şirket **Kanada'ya genişler**. Gereken değişiklikler: yeni satış bölgeleri eklenmeli; Kanada vergileri ve farklı iş kuralları desteklenmeli; iki para birimi (USD ve CAD) desteklenmeli. Bu değişiklikler şirket için harika, yazılım için felaket olabilir.

- **Moving target problem:** *A change in the requirements while the software product is being developed.* Sebep iyi olsa bile ürün olumsuz etkilenir: modüller arasında bağımlılıklar (dependencies) oluşur.
- Ürüne yapılan her değişiklik potansiyel olarak bir **regression fault**'a (görünüşte ilgisiz bir bölümde ortaya çıkan hataya) yol açabilir.
- Çok fazla değişiklik olursa ürünün tamamı yeniden tasarlanıp yeniden gerçekleştirilmek zorunda kalınabilir.
- Büyüyen şirketler her zaman değişir; değişikliği isteyen kişinin yeterli gücü (clout) varsa yapılacak bir şey yoktur. **Moving target probleminin bir çözümü yoktur**; sadece etkisi azaltılabilir.

2.4 Yineleme ve artırım (Iteration and Incrementation)

2.4.1 Miller yasası ve stepwise refinement

Miller's Law: İnsan aynı anda yaklaşık **yedi parça (chunk) bilgiye** odaklanabilir. Daha fazla bilgiyle başa çıkmak için **stepwise refinement** kullanılır:

- O an en önemli olan konulara odaklan;
- Şu an daha az kritik olanları ertele;
- Sonunda her konu ele alınır, ama **önem sırasına göre**.

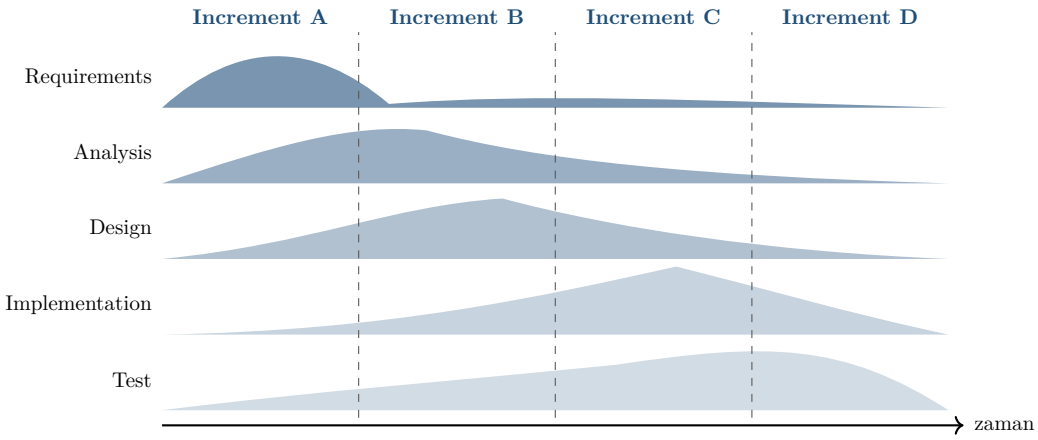
Bu, **artırımlı (incremental)** bir süreçtir. Hoca bunu *snowball effect* (kartopu etkisi) diye anlatır: önce çekirdek (en öncelikli) kısım yapılır, sonra her artırımda yeni özellikler eklenerek ürün büyür.

2.4.2 Iteration ve incrementation

- **Iteration (yineleme):** Temel geliştirme süreci yinelemelidir; *her ardışık sürüm hedefe bir öncekinden daha yakındır.* Bir artırım içinde, hedefe ulaşılana kadar aynı iş akışları tekrarlanır.
- **Incrementation (artırım):** Ürün parçalara (artırımlara) bölünür; her artırımda ürüne yeni işlevler eklenir.
- İkisi **birlikte** kullanılır: tek bir "requirements phase" veya "design phase" yoktur; her fazın birden çok örneği vardır. Artırım sayısı dört olmak zorunda değildir; yineleme sayısı da üç olmak zorunda değildir.

2.4.3 Klasik fazlar ve iş akışları (workflows)

Gerçek dünyada ardışık fazlar yoktur. Bunun yerine **beş çekirdek iş akışı** bütün yaşam döngüsü boyunca yürütülür: **requirements, analysis, design, implementation, test workflow**. Her an bir iş akışı baskındır: başta requirements, sonda implementation ve test. Planlama ve dokümantasyon bütün yaşam döngüsü boyunca yapılır.



Şekilde her iş akışının alanı, o artırımda o iş akışına harcanan emeği gösterir (kitaptaki Figure 2.4'ün sadeleştirilmiş hâli).

2.4.4 Winburg yeniden: artırımlar ve bakım

Evolution-tree modeli yinelemeli-artırımlı modelin üzerine oturtulursa: her bölüm (episode) bir **artırıma** karşılık gelir; her artırım her iş akışını içermek zorunda değildir; Episode 2 ve 3 **corrective**, Episode 4 **perfective** bakımdır.

2.5 Yinelemeli-artırımlı modelin riskleri ve güçlü yanları

Proje bir dizi **mini projeden** (artırımdan) oluşur. Her mini projede artifact'lar genişletilir (incrementation), kontrol edilir (test workflow) ve gerekirse değiştirilir (iteration). **Her yineleme küçük ama tam bir waterfall modelidir.**

Güçlü yanları:

1. Ürünün doğruluğunu kontrol etmek için **birçok fırsat** vardır; her yineleme test iş akışını içerir, hatalar erken bulunur.
2. Mimarinin **sağlamlığı (robustness)** yaşam döngüsünün başında belirlenebilir. *Architecture:* bileşen modüller ve birbirine nasıl oturdukları. *Robustness:* dağılmadan genişlemeyi ve değişikliği kaldırabilme özelliği.

3. **Riskler erken azaltılabilir** (mitigate).
4. Baştan itibaren **çalışan bir sürüm** vardır; müşteri ve kullanıcılar deneyerek ne değişmesi gerektiğini görür. Kısmi sürümler teslim edilerek yeni ürüne geçiş yumuşatılır.
5. Modelin işe yaradığına dair **deneysel kanıt** vardır: Standish Group CHAOS raporlarında (1994–2006) başarılı proje oranı artmıştır.

Yönetimi: Yinelemeli-artırımlı model de waterfall kadar disiplinlidir, çünkü **art arda uygulanan waterfall modelidir**.

2.6 Diğer yaşam döngüsü modelleri (kısaca)

Kitapta karşılaştırılan modeller: **code-and-fix** (tasarım/spesifikasyon yok, kodla-düzelt; küçük programlar dışında kötü), **waterfall** (belge güdümlü, geri besleme döngülü doğrusal model), **rapid prototyping** (müşteriye hızlıca prototip gösterip gereksinimi netleştirme), **open-source**, **agile processes** (kısa iterasyonlar, sürekli müşteri teması), **synchronize-and-stabilize** (Microsoft modeli; her gün bileşenler birleştirilip test edilir), **spiral** (her döngüde **risk analizi**; odak risklerdir). Hoca bu modellerin ayrıntısına girmediğini söyledi; isimlerini ve birer cümlelik özlerini bilmek yeterlidir.

2.7 Stable Intermediate Forms (kararlı ara formlar)

! Hoca derste vurguladı 2025 dersi ve WhatsApp

Hoca bu konuyu *Moving Target Problem* slaytı (Slide 2.43) açıkken not defterine şöyle yazdı: `Stable Intermediate Forms: -Three Points of Contact; -Copy before release; -Iterative and Incremental Software Development (stepwise refinement, snowball effect)`. WhatsApp: *"stable intermediate forms için de sorarım demişti"*, *"Stable İntermediate Forms'un açıklamalarını isterim dedi."* 2024–25 ve 2025–26 vizelerinin ikisinde de soruldu (15 puan, açıklamalarıyla).

Fikir: Karmaşık bir sistem, **her adımda kararlı (çalışan, tutarlı) ara formlardan** geçerek gelişirse çok daha hızlı ve güvenli gelişir. Bir değişiklik ters giderse en son kararlı forma geri dönülür; her şey baştan yapılmaz. Bu, moving target problemi ve kaçınılmaz hataların etkisini azaltmanın yoludur. Hoca üç yöntem saydı:

1. Three Points of Contact (üç temas noktası)

Kaya tırmanışı / tepe tırmanma (*climbing a hill*) benzetmesidir: Tırmanıcının dört uzvundan **üçü her zaman sabit** bir tutamaktadır, **yalnızca biri** hareket eder. Böylece bir tutamak kayarsa düşmez, sabit üç noktaya geri döner.

Yazılımda: **Aynı anda yalnızca tek bir şey değiştirilir, diğer her şey sabit tutulur.** Değişiklik test edilip kararlı hâle gelince sıradaki değişikliğe geçilir. Değişiklik olumsuz sonuç verirse geri dönmek kolaydır ve hatanın kaynağı bellidir (tek değişiklik yapıldığı için).

Sınava yazılabilecek açıklama

Three points of contact: like a climber who keeps three limbs fixed and moves only one, we change only one thing at a time and keep everything else stable. After the change is tested and becomes stable we change the next thing. If the change fails, we can easily go back to the last stable form and we know which change caused the problem.

2. Copy Before Release (yayından önce kopyala)

Yeni bir sürüm yayımlanmadan / değişiklik canlıya alınmadan önce **mevcut kararlı sürümün kopyası (yedeği) alınır**. Yeni sürümde sorun çıkarsa hemen kararlı kopyaya dönülür. Sürüm kontrolü (version control) ve *baseline* kavramı bunun uygulamasıdır.

Sınava yazılabilecek açıklama

Copy before release: before releasing a new version, a copy (backup) of the current stable version is taken. If the new release causes problems, we can roll back to the stable copy. Each copy is a stable intermediate form (baseline).

3. Iterative and Incremental Software Development

Yazılım tek seferde değil, **tekrarlanan döngüler (iteration)** ve **küçük parça eklemeleri (incrementation)** ile geliştirilir. Her artırımın sonunda *çalışan ama eksik* bir ürün vardır; bu her biri kararlı bir ara formdur. Önce en önemli çekirdek yapılır (**stepwise refinement**, Miller yasası), sonra ürün kartopu gibi büyür (**snowball effect**). Riskler azalır, yeni bilgiyle evrimsel gelişim sağlanır.

Sınava yazılabilecek açıklama

Iterative and incremental development: the product is divided into increments (mini projects). In each increment the workflows are iterated until the target is reached, so at the end of every increment we have a working but incomplete product, i.e. a stable intermediate form. We start with the most important core (stepwise refinement, Miller's law) and grow the product step by step like a snowball (snowball effect).

Sık yapılan hatalar

- Sadece başlıkları yazmak yeterli değil; hoca **açıklamalarını** istiyor.
- "Stable intermediate *terms*" değil **forms**. Bazı öğrenci notlarında "terms" yazıyor; doğrusu *forms*.
- WhatsApp'ta Türkçe cevabın geçen yıl kabul edilmediği söyleniyor. Açıklamaları İngilizce yazmaya çalışın; zorlandığınız yerde kısa Türkçe ekleme yapılabilir (hoca: "zorlandığınız kısımda azıcık yazabilirsiniz").

2.8 Kısa tanım soruları

Moving target problem A change in the requirements while the software product is being developed. It induces dependencies and can cause regression faults. There is no solution; its effects can only be minimized.

Regression fault A fault in an apparently unrelated part of the software caused by a change.

Miller's law At any one time we can concentrate on only about seven chunks of information.

Stepwise refinement Concentrate on the currently most important aspects and postpone the less critical ones; every aspect is eventually handled in order of importance.

Iteration Each successive version is closer to the target than its predecessor.

Incrementation Adding new functionality step by step; the product is built as a set of increments (mini projects).

Baseline A complete set of artifacts at a point in time (e.g. at the end of an episode).

Robustness The property of being able to handle extensions and changes without falling apart.

3 Unified Process (Chapter 3)

! Hoca derste vurguladı Hafta 1 ve 4

"One of the most important chapters is third chapter because the unified process is described here ... These phases and workflows of unified process is very important in terms of our course." Eski vize notlarında: *"Unified Process nedir? Fazlarını ve iş akışlarını açıklayınız — bunları sınavda soruyorum."* IEEE SPMP için: *"Bu formatı önemsiyorum ve genelde sınavlarda da soruyorum, başlıklarını soruyorum ... 1, 2, 3, 4, 5, 6, 7, 8. ana başlıkları."*

3.1 Unified Process nedir?

1990'ların en başarılı üç nesne yönelimli yöntemi Booch'un yöntemi, Jacobson'un Objectory'si ve Rumbaugh'un OMT'siydi. 1999'da üçü bu yöntemleri birleştirerek tam bir nesne yönelimli analiz ve tasarım metodolojisi yayımladı: önce *Rational Unified Process (RUP)*, sonra *Unified Software Development Process (USDP)*, bugün kısaca **Unified Process**.

- Unified Process **nesne yönelimli bir yazılım geliştirme metodolojisidir**.
- Bir yazılım üretmek için **bir dizi adım değildir**; "herkese uyan tek" bir metodoloji olamaz, çünkü çok farklı türde yazılımlar vardır.
- **Uyarlanabilir (adaptable)** bir metodolojidir: geliştirilecek ürüne göre değiştirilmelidir.
- Bir **modelleme tekniğidir**: model, ürünün çeşitli yönlerini gösteren UML diyagramları kümesidir. **UML (Unified Modeling Language)** hedef ürünü modellemek için kullanılan araçtır. UML grafiktir: "bir resim bin kelimeye bedeldir"; yazılımcıların hızlı ve doğru iletişim kurmasını sağlar.
- Nesne yönelimli paradigma doğası gereği **yinelemeli ve artırımlıdır**: UML diyagramları tatmin edici olana kadar tekrar tekrar yineleme ve artırım yapılır.

Sınav cevabı kalıbı: "What is the Unified Process?"

The Unified Process is an object-oriented software development methodology (Booch, Jacobson, Rumbaugh, 1999). It is not a series of steps for constructing a software product; it is an adaptable methodology that has to be modified for the specific product. It is a modeling technique: the product is represented by UML diagrams. It is iterative and incremental and two-dimensional: it has five core workflows (technical context) and four phases (business context).

3.2 Beş çekirdek iş akışı (core workflows)

3.2.1 Requirements workflow

Amaç: müşterinin *ihtiyaçlarını* (needs) belirlemek; *istediklerini* (wants) değil.

1. **Uygulama alanını (application domain) anla**: ürünün çalışacağı iş ortamını öğren.
2. **İş modelini (business model) kur**: müşterinin iş süreçlerini UML ile tanımla. Müşteri herhangi bir anda maliyeti haklı bulmazsa geliştirme hemen durur.
3. Müşterinin **kısıtlarını (constraints)** belirle: *deadline* (teslim tarihi; ürünler çoğu zaman *mission critical*), *parallel running* (eski sistemle paralel çalışma), *portability* (taşınabilirlik), *reliability* (güvenilirlik), *rapid response time* (hızlı yanıt), *cost* (müşteri bütçeyi nadiren söyler; teklif / ihale usulü kullanılır).

Bu kavram keşfinin (*concept exploration*) amacı müşterinin neye **ihtiyaç** duyduğunu bulmaktır.

3.2.2 Analysis workflow

Amaç: gereksinimleri **analiz etmek ve inceltmek (refine)**. Neden requirements'ta yapılmıyor? Çünkü requirements artifact'ları **müşterinin tamamen anlayacağı doğal dilde** olmak zorundadır ve bütün doğal diller belirsizdir. Örnek: "Veritabanından bir parça kaydı ve bir fabrika kaydı okunur. *Eğer* A harfinin hemen ardından Q harfini *içeriyorsa*, o parçanın o fabrikaya taşıma maliyetini hesapla." — "içeriyorsa"nın öznesi ne: parça kaydı mı, fabrika kaydı mı, veritabanı mı?

Bu yüzden iki ayrı iş akışı gerekir: requirements artifact'ları **müşterinin dilinde**, analysis artifact'ları ise tasarımcılar için **kesin ve yeterince tam** olmalıdır.

- **Specification document** bir **sözleşmedir (contract)**. "Optimal", "%98 tamamlanmış" gibi belirsiz ifadeler içermemelidir. **Çelişki (contradictions), eksiklik (omissions) ve tamamlanmamışlık (incompleteness)** olmamalıdır. Tam ve doğru spesifikasyon **test** ve **bakım** için şarttır.
- Müşteri spesifikasyonu imzaladıktan sonra ayrıntılı planlama ve kestirim başlar: **SPMP** (software project management plan) hazırlanır: **maliyet kestirimi (cost estimate), süre kestirimi (duration estimate), teslim edilecekler (deliverables), kilometre taşları (milestones), bütçe (budget)**. Bu, SPMP için **mümkün olan en erken zamandır**.

3.2.3 Design workflow

Amaç: analiz iş akışını, programcıların gerçekleştirebileceği forma gelene kadar inceltmek. Birçok **nonfunctional requirement** bu aşamada kesinleşir: programlama dili seçimi, yeniden kullanım, taşınabilirlik.

- Klasik: **architectural design** (ürünü modüllere ayırma) ve **detailed design** (her modülün veri yapıları ve algoritmaları).
- Nesne yönelimli: sınıflar **analysis** iş akışında çıkarılır, **design** iş akışında tasarlanır. Klasik mimari tasarım nesne yönelimli analizin, klasik ayrıntılı tasarım nesne yönelimli tasarımın bir parçasına karşılık gelir.
- **Tasarım kararları saklanmalıdır**: çıkmaz sokağa girildiğinde geri dönebilmek ve bakım ekibinin "tekerleği yeniden icat etmemesi" için.

3.2.4 Implementation workflow

Amaç: hedef ürünü seçilen programlama dilinde gerçekleştirmek. Büyük ürün alt sistemlere (subsystems), alt sistemler bileşenlere / kod artifact'larına (components, code artifacts) bölünür.

3.2.5 Test workflow

Test, her geliştiricinin, bakımcının ve **kalite güvence (SQA) grubunun** sorumluluğudur. **İzlenebilirlik (traceability)** başarılı testin önemli gereksinimidir: analysis artifact'larındaki her öğe requirements artifact'larındaki bir öğeye izlenebilmelidir (design ve implementation için de aynısı).

- **Requirements / analysis artifact'ları:** *review* (gözden geçirme) ile kontrol edilir; müşteri temsilcisi ve analiz ekibi bulunur. SPMP de benzer şekilde kontrol edilir (özellikle maliyet ve süre kestirimleri).
- **Design artifact'ları:** design review şarttır; müşteri temsilcisi genellikle bulunmaz.
- **Implementation artifact'ları:**
  - **Unit testing**: her bileşen gerçekleştirildiği anda test edilir.

- **Integration testing**: her yinelemenin sonunda bileşenler birleştirilip test edilir.
- **Product testing**: ürün tamamlanmış göründüğünde bütün olarak test edilir.
- **Acceptance testing**: ürün müşterinin bilgisayarına kurulduktan sonra **müşteri** test eder.
- COTS yazılımlarda: **alpha release** (geliştirme testleri sürerken olası müşterilere test için verilen sürüm) ve **beta release** (neredeyse tamamlanmış, ilk işletme kalitesindeki sürüm). Alfa/beta sitesi olmanın avantajı ürünü erken görmek ve rakiplerden önce davranmak; dezavantajı hatalı yazılımla uğraşmaktır.

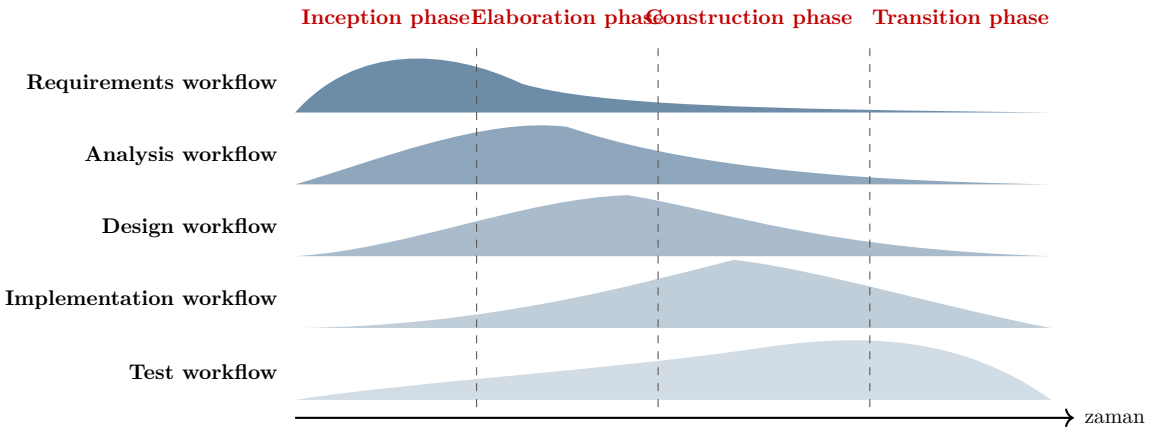
3.2.6 Postdelivery maintenance ve retirement

Teslim sonrası bakım geliştirmenin vazgeçilmez parçasıdır; **diğer bütün etkinliklerin toplamından fazla para** buna harcanır. Sorunların başlıca kaynağı her türlü **dokümantasyon eksikliğidir**. İki tür test gerekir: yapılan değişikliğin testi ve **regression testing** (bu yüzden önceki bütün test durumları ve beklenen sonuçları saklanmalıdır).

Yazılım şu durumlarda bakım yapılamaz hâle gelebilir: tasarımda köklü değişiklik; tamamen yeni donanım/işletim sistemi; eksik veya yanlış dokümantasyon; donanım değişikliği yüzünden sıfırdan yazmanın daha ucuz olması. Bunlar da aslında bakımdır (mevcut yazılımın yeniden yazılması). **Gerçek retirement** nadirdir: müşteri kurumun ürünün sağladığı işleve **artık hiç ihtiyacı kalmadığında** olur.

3.3 Unified Process'in dört fazı

Unified Process'te artırımlar **faz** olarak adlandırılır. Teoride herhangi sayıda artırım olabilir; pratikte geliştirme dört artırımdan oluşur:



Unified Process'teki her adım hem beş iş akışından birine hem de dört fazdan birine düşer. Neden iki kez düşünülür? **Workflow = adımın teknik bağlamı (technical context), phase = adımın iş/ekonomik bağlamı (business context)**.

3.3.1 Inception phase (başlangıç fazı)

Amaç: önerilen yazılım ürününün **ekonomik olarak uygulanabilir (economically viable)** olup olmadığını belirlemek. Adımlar:

1. Alanı anla (understand the domain).
2. İş modelini kur (build the business model).

3. Projenin kapsamını daralt (delimit the scope): iş modelinin yazılımın kapsadığı alt kümesine odaklan.
4. İlk **business case**'i hazırlamaya başla. Cevaplanacak sorular: Ürün maliyet-etkin mi? Yatırımın geri dönüşü ne kadar sürer? Geliştirmezsek maliyeti ne olur? Pazarlanacaksa pazar araştırması yapıldı mı? Zamanında teslim edilebilir mi? Geç kalırsa etkisi ne olur?

Riskler ve nasıl azaltılacakları da belirlenir. **Üç ana risk kategorisi:**

1. **Technical risks** (ekip deneyimi, yeni donanım zamanında gelir mi, CASE araçları var mı),
2. **Gereksinimleri doğru belirleyememe riski** (requirements workflow doğru yapılarak azaltılır),
3. **Mimariyi doğru kuramama riski** (mimari yeterince sağlam olmayabilir).

Riskler **sıralanır**; kritik riskler önce azaltılır. Inception fazında az miktarda analiz ve tasarım yapılabilir; kodlama genellikle yapılmaz ama bazen **proof-of-concept prototype** yapılır. Test iş akışı fazın neredeyse başında başlar (gereksinimlerin doğruluğu). Başta tüm projeyi planlamaya yetecek bilgi olmadığından **yalnızca bir sonraki fazın (elaboration) planı** yapılır.

Inception çıktıları (deliverables): domain model'in ilk sürümü, business model'in ilk sürümü, requirements artifact'larının ilk sürümü, analysis artifact'larının ön sürümü, mimarinin ön sürümü, **ilk risk listesi**, use case'lerin ilk sıralaması, elaboration fazının planı, **business case'in ilk sürümü** (fazın asıl amacı).

3.3.2 Elaboration phase (ayrıntılandırma fazı)

Amaç: ilk gereksinimleri **inceltmek**; mimariyi inceltmek; riskleri izleyip önceliklerini güncellemek; business case'i inceltmek; **project management plan**'ı (SPMP) hazırlamak. Bu fazda requirements iş akışı neredeyse tamamlanır, analiz iş akışının neredeyse tamamı yapılır, mimari tasarıma başlanır.

Elaboration çıktıları: tamamlanmış domain model, business model, requirements ve analysis artifact'ları; mimarinin güncel sürümü; güncel risk listesi; **project management plan** (projenin geri kalanı için); tamamlanmış business case.

3.3.3 Construction phase (yapım fazı)

Amaç: ürünün **ilk işletme kalitesindeki (first operational-quality) sürümünü** üretmek; buna **beta release** denir. Vurgu **implementation ve test**tedir: modüllerin unit testi, alt sistemlerin integration testi, bütün sistemin product testi.

Construction çıktıları: ilk kullanıcı kılavuzu ve diğer kılavuzlar; bütün artifact'lar (beta sürümleri); tamamlanmış mimari; güncel risk listesi; güncel project management plan; gerekirse güncel business case.

3.3.4 Transition phase (geçiş fazı)

Amaç: müşterinin gereksinimlerinin **gerçekten karşılandığından emin olmak**. Hatalar düzeltilir, bütün kılavuzlar tamamlanır, daha önce fark edilmemiş riskler bulunmaya çalışılır. Bu faz, beta sürümün kurulduğu sitelerden gelen **geri bildirimle** yürür.

Transition çıktıları: bütün artifact'ların son sürümleri, tamamlanmış kılavuzlar.

Faz	Amaç	Ağırlıklı iş akışı
Inception	Ürün ekonomik olarak yapılabilir mi? İlk business case.	Requirements
Elaboration	Gereksinimleri ve mimariyi incelt, SPMP'yi çıkar.	Requirements + Analysis
Construction	İlk işletme kalitesinde sürüm (beta).	Implementation + Test
Transition	Müşteri gereksinimleri karşılandı mı? Hataları düzelt, kılavuzları tamamla.	Test (geri bildirim)

Çıkmış test sorusu

"Which one is NOT a phase of Unified Process?" (Elaboration, Construction, **Testing**, Transition, Inception). Cevap: **Testing**. Test bir *iş akışıdır*, faz değildir. Fazlar ile iş akışlarını karıştırmayın.

3.4 Tek ve iki boyutlu yaşam döngüsü modelleri

- Geleneksel yaşam döngüsü (ör. waterfall) **tek boyutludur**: sadece fazlar ekseni vardır.
- Unified Process **iki boyutludur**: bir eksende iş akışları (teknik bağlam), diğerinde fazlar (iş bağlamı). Evolution-tree modeli de iki boyutludur.
- Neden bu karmaşıklık gerekli? İdeal dünyada her iş akışı bir sonrakinden önce biterdi. Gerçekte iş çok büyüktür; **Miller yasası** gereği iş artırımlara (fazlara) bölünmeli ve her artırımda iş bitene kadar yineleme yapılmalıdır. Başlangıçta gereksinimleri tamamlamaya yetecek bilgi yoktur. Ürün alt sistemlere, onlar da bileşenlere bölünür.
- Unified Process kaçınılmaz değişiklikleri (**moving target problem**, **kaçınılmaz hatalar**) iyi karşılar; onları *ortadan kaldırmaz*, *etkilerini azaltır*. Büyük bir problemi küçük, büyük ölçüde bağımsız alt problemler kümesi olarak ele almanın bugüne kadarki en iyi çözümüdür; gelecekte daha iyisiyle yer değiştirecektir.

"Why do we need a two-dimensional model?"

Real projects are too large to be completed in one pass. Because of Miller's law the task is divided into increments (phases) and within each increment the workflows are iterated. A two-dimensional model shows both the workflows (technical context) and the phases (business context); it handles the moving target problem and the inevitable mistakes better, and faults are observed earlier.

3.5 IEEE Software Project Management Plan (SPMP)

SPMP'nin özü (Chapter 3): **cost estimate, duration estimate, deliverables, milestones, budget**. IEEE Std 1058 bunun ayrıntılı bir çerçevesini verir (kitapta Section 9.5, Figure 9.9). Hoca **yalnızca 8 ana başlığı** sorar:

#	Ana başlık	Alt başlıklar / içerik
1	**Overview**	1.1 Project summary (purpose, scope and objectives; assumptions and constraints; project deliverables; schedule and budget summary), 1.2 Evolution of the project management plan
2	**Reference materials**	Uyulan dokümanlar ve standartlar
3	**Definitions and acronyms**	Tanımlar ve kısaltmalar (ör. MSG)
4	**Project organization**	External interfaces, internal structure, roles and responsibilities
5	**Managerial process plans**	Start-up plan (estimation, staffing, resource acquisition, staff training), work plan (work activities, schedule/resource/budget allocation), control plan (requirements, schedule, budget, quality control; reporting; metrics collection), risk management plan, project closeout plan
6	**Technical process plans**	Process model, methods/tools/techniques, infrastructure plan, product acceptance plan
7	**Supporting process plans**	Configuration management, testing, documentation, quality assurance, reviews and audits, problem resolution, subcontractor management, process improvement
8	**Additional plans**	Projeye özel ek planlar

Ezber ipucu

"**O**verview – **R**eference – **D**efinitions – **O**rganization – **M**anagerial – **T**echnical – **S**upporting – **A**dditional": önce proje tanıtılır (1–3), sonra kim yapacak (4), sonra nasıl yönetilecek (5), teknik olarak nasıl yapılacak (6), destek süreçleri (7), ekler (8). IEEE çerçevesi her tür ürün için tasarlanmıştır, belirli bir yaşam döngüsü modeli dayatmaz ve Unified Process için idealdir (requirements control ve risk management bölümleri vardır).

Eski sürüm: 5 başlıklı SPMP

Klasördeki 2021 öğrenci notunda IEEE SPMP'nin **eski sürümü** (IEEE 1058.1-1987, kitabın eski baskıları) yazılı: **1 Introduction** (project overview, deliverables, evolution of the SPMP, reference materials, definitions and acronyms), **2 Project Organization** (process model, organizational structure, boundaries and interfaces, project responsibilities), **3 Managerial Process** (management objectives and priorities; assumptions, dependencies and constraints; risk management; monitoring and controlling mechanisms; staffing plan), **4 Technical Process** (methods, tools and techniques; software documentation; project support functions), **5 Work Packages, Schedule and Budget**. Hoca videoda kitabın 8. baskısındaki **8 başlıklı** çerçeveyi gösterdi; sınavda 8 başlığı yazın.

3.6 Yazılım sürecini iyileştirme (Improving the software process)

ABD Savunma Bakanlığı girişimiyle kurulan **Software Engineering Institute (SEI)**'ye göre yazılımdaki temel sorun **sürecin kötü yönetilmesidir**. Hocanın vurgusu: *süreç kaliteliyse ürün de kaliteli olur*; bu standartlar ürünle değil **üretim süreciyle** ilgilenir, her şeyin belgelenmesini ister: *belge yoksa ilerlemeyi izleyemezsin; izleyemezsen başarını ölçemezsin; ölçemezsen kendini geliştiremezsin.*

- **CMM (Capability Maturity Model)** beş olgunluk seviyesi: **1 Initial** (ad hoc, kriz yönetimi), **2 Repeatable** (temel proje yönetimi; önceki benzer projeler tekrarlanabilir), **3 Defined** (süreç tamamen belgelenmiş), **4 Managed** (nitel ve nicel hedefler, ölçüm), **5 Optimizing** (sürekli süreç iyileştirme).
- **ISO 9000 serisi**: kalite sistemleri standardı (yazılım için ISO 9000-3).
- **ISO/IEC 15504 (SPICE)**: süreç değerlendirme standardı.

- Hoca: *"you can read it but the ISO 25000 series are more important for me"* ⇒ ayrıntı için Konu 5 (ISO/IEC 25010).

3.7 Kısa tanım soruları

Workflow vs phase A workflow is the technical context of a step; a phase is the business context of a step. Every step falls into one workflow and one phase.

Business case A document describing the scope of the product and its financial details (revenue projections, cost estimates or cost–benefit analysis); its initial version is the overall aim of the inception phase.

Beta release The first operational-quality version produced at the end of the construction phase.

Traceability Every item in an artifact must be traceable to an item in the previous artifact.

Specification document A contract between client and developer; it must not contain contradictions, omissions or incompleteness.

Three major risks Technical risks, the risk of not getting the requirements right, and the risk of not getting the architecture right.

4 Modüllerden Nesnelere: Cohesion ve Coupling (Chapter 7)

! Hoca derste vurguladı Hafta 1, 4 ve 5

"The most important chapter is chapter seven. It describes how modules should be decomposed." Eski vize notu: *"Cohesion (7) ve Coupling (5)'i açıklayınız. Türlerinin başlıklarını yazınız."* 2025 WhatsApp: *"Derste kodu verip hangi türde cohesion ya da coupling olduğunu sorarım demişti"*, *"vizede veriyor ama başka bir seviyeye dönüştürme isteyebilir"*. Son iki vizede bu sorudan **kod verilip tür bulma ve daha iyi hâle getirme** istendi.

4.1 Neden modülerleştirme?

Hocanın sorusu: "Kitaplarını rafta neden kategorilere ayırırsın?" Yazılımı alt modüllere bölmek **yönetimi** ve **bakımı** kolaylaştırır (*divide and manage*): hata kolay bulunur, regression etkisi olmadan düzeltilir. Ama **her modülerleştirme iyi tasarım değildir.** İyi bir yazılım için iki şey gerekir: (1) iyi bir yaşam döngüsü modeli (**iterative and incremental**) ve (2) **robust architecture**: **high cohesion, low coupling** ve SOLID ilkeleri.

4.2 Modül nedir?

Module: *A lexically contiguous sequence of program statements, bounded by boundary elements, with an aggregate identifier.*

- **Lexically contiguous:** kodda bitişik (ardışık) satırlar.
- **Boundary elements:** sınır öğeleri; C tabanlı dillerde `{ ... }`, Pascal'da `begin ... end`.
- **Aggregate identifier:** modülün tamamına verilen **isim**.

Hocanın derste sorduğu soru

`if (kosul) { s1; s2; } else { s3; s4; }` bloğu bir modül müdür? **Hayır.** Ardışık ve sınır öğeleriyle çevrili ama **adı (aggregate identifier) yok**; ismiyle çağrılamaz. Bir metodun içine konup metoda isim verilirse modül olur. Metotlar, sınıflar, struct'lar modüldür.

4.2.1 Bilgisayar tasarımı örneği

Beceriksiz bir mimar ALU, shifter ve 16 register'dan oluşan bir bilgisayarı AND, OR, NOT kapılarıyla kuracaktır.

- **Tasarım 1:** işleve göre üç çip: register'lar bir çipte, ALU bir çipte, shifter bir çipte.
- **Tasarım 2:** kapı türüne göre üç çip: bütün AND kapıları bir çipte, OR'lar bir çipte, NOT'lar bir çipte.

İkisi işlevsel olarak eşdeğerdir ama tasarım 2: **anlaşılması zor**, **hatanın yerini bulmak zor** (ALU çıktısı yanlışsa hangi kapı?), **genişletmesi zor** (belleği büyütmek için her çipe kapı ekleyip bağlantıları yeniden kurmak gerekir), **başka üründe yeniden kullanılamaz**. Modüller tasarım 1 gibi olmalıdır:

maximal relationships within modules, minimal relationships between modules

- **Cohesion:** bir modülün **içindeki** etkileşimin derecesi (*degree of interaction within a module*). **Yüksek** olmalı.

- **Coupling: iki modül arasındaki** etkileşimin derecesi (*degree of interaction between two modules*). **Düşük** olmalı.

Hocanın uyarısı

"Our friends are confused about cohesion and coupling. Cohesion is within a module. Coupling is between modules." Kohezyon **tek bir modülün** içini, coupling **iki modülün** ilişkisini anlatır. Sınavda "robust architecture" sorusunun cevabı: **loose (low) coupling / high cohesion**.

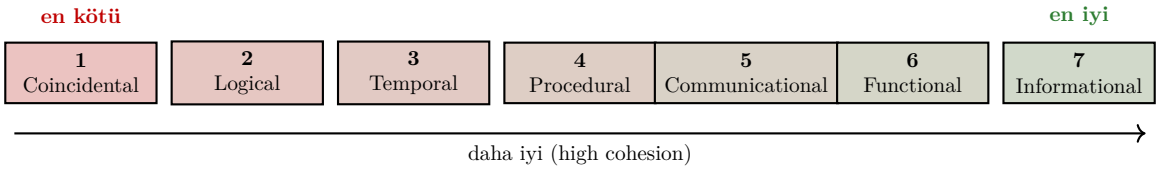
4.2.2 Bir modülün function, logic ve context'i

Örnek: "Newton algoritmasıyla double precision sayıların karekökünü hesaplayan modül."

- **Function** (ne yapar): karekök hesaplar.
- **Logic** (nasıl yapar): Newton algoritmasıyla.
- **Context** (ne üzerinde çalışır): double precision sayılar.

Klasik (C/SD) yaklaşımda modülün adı işlevidir: `compute_square_root` (alt çizgili Pascal notasyonu); nesne yönelimli dillerde `ComputeSquareRoot` (camel/Pascal case) kullanılır.

4.3 Cohesion (modül içi uyum): 7 seviye



Ölçek doğrusal değildir (non-linear). Kitapta 6 ve 7 aynı seviyede gösterilir; hoca 7'yi (informational) en iyi olarak sayar.

1. Coincidental cohesion (tesadüfi) — en kötü

Tanım: Modül **birbiriyle tamamen ilgisiz** birden çok eylem yapar.

Kitap örneği: `print_next_line, reverse_string_of_characters_comprising_second_parameter, add_7_to_fifth_parameter, convert_fourth_parameter_to_floating_point`.

Nasıl oluşur? "Her modül 35 ile 50 satır arasında olacak" gibi kurallardan: uzun modüller bölünür, kısa parçalar başka modüllere eklenir.

Neden kötü? Bakımı zorlaştırır (tasarım 2 gibi spagetti), **yeniden kullanılamaz**.

Düzeltme (kolay): modülü her biri tek iş yapan ayrı modüllere böl (SOLID'in **S**'si: Single Responsibility).

```
void Islemler(string s, ref int sayac, string x)
{
    Console.WriteLine("Sonraki satir");          // yazdirma
    char[] d = s.ToCharArray(); Array.Reverse(d); // string ters cevirme
    sayac += 7;                                   // sayaca 7 ekleme
    double sonuc = Convert.ToDouble(x);           // ondaliga cevirme
}
```

2. Logical cohesion (mantıksal)

Tanım: Modül **birbiriyle ilgili** bir dizi eylem yapar; bunlardan **biri çağıran modül tarafından seçilir** (opCode / flag / switch).

```
void Rapor(int opCode)          // cagiran, hangi raporun basilacagini secer
{
    if (opCode == 1)      YerelSatisRaporuYazdir();
    else if (opCode == 2) BolgeselSatisRaporuYazdir();
    else if (opCode == 3) UlusalSatisRaporuYazdir();
    else                  UluslararasiSatisRaporuYazdir();
}
```

Kitap örnekleri: `function_code = 7; new_operation(op_code, dummy_1, dummy_2, dummy_3);` (dummy değişkenler kullanılmıyor); bütün giriş/çıkışı yapan bir nesne; OS/VS2'de 13 farklı eylem yapan, arayüzünde 21 parametre bulunan modül.

Neden kötü? (1) **Arayüz anlaşılması zor** (hangi opCode'da hangi parametre kullanılıyor?), (2) birden fazla eylemin kodu **iç içe geçebilir (intertwined)**, (3) yeniden kullanmak zordur. Hoca: yeni bir lazer yazıcı eklemek için `if (opCode == 38)` eklemek gerekir; bu **Open-Closed ilkesini** ihlal eder ("genişletmek için mevcut kodu değiştiriyoruz"). Kitap: yeni bir teyp birimi eklenince lazer yazıcı kodu etkilenebilir.

Düzeltme: her eylemi kendi metoduna ayır (functional); çağıran doğrudan ilgili metodu çağırsın. Nesne yönelimli çözüm: ortak bir `IRapor` arayüzü ve her rapor için ayrı sınıf (polymorphism); yeni rapor eklemek mevcut kodu değiştirmez.

3. Temporal cohesion (zamansal)

Tanım: Modül **zaman açısından ilişkili** bir dizi eylem yapar: sadece *aynı anda / art arda yapılmaları gerektiği için* bir aradadırlar (tipik: başlatma ve kapanış işlemleri).

Kitap örneği (`perform_initialization`): `open_old_master_file, new_master_file, transaction_file, print_file; initialize_sales_district_table; read_first_transaction_record; read_first_old_master_record.`

```
void ProgramiBaslat()
{
    dosya = File.Open("ana.dat", FileMode.Open); // dosya ac
    satisBolgeTablosu.Clear();                   // tabloyu sifirla
    ilkKayit = OkuIlkKayit();                    // ilk kaydi oku
    Console.WriteLine(DateTime.Now);             // zamani yaz
}
```

Neden kötü? Eylemler birbirleriyle **zayıf**, ama **başka modüllerdeki** eylemlerle **güçlü** ilişkilidir (ör. `sales_district_table` başka modüllerde güncellenir; tablo değişirse bu modül de değişmelidir). Yeniden kullanılamaz.

Düzeltme: her başlatma işini ilgili modüle/sınıfa taşı (tablo başlatma tablo sınıfının constructor'ına, dosya açma dosya sınıfına).

4. Procedural cohesion (işlemsel)

Tanım: Modül, **ürünün izlediği prosedürle (iş akışıyla) ilişkili** bir dizi eylem yapar. Kitap örneği: `read_part_number_and_update_repair_record_on_master_file`.

```
void ParcaNoOkuVeTamirKaydiGuncelle()
{
    int parcaNo = int.Parse(Console.ReadLine()); // parca numarasini oku
    TamirKaydi k = anaDosya.Bul(parcaNo);        // farkli veri: tamir kaydi
    k.TamirSayisi++;
    anaDosya.Guncelle(k);
}
```

Temporal'dan iyidir (eylemler ürünün işleyişi açısından ilişkili), ama eylemler hâlâ zayıf bağlıdır, modül birden çok iş yapar, **yeniden kullanılamaz**.

Düzeltme: `ParcaNoOku()` ve `TamirKaydiGuncelle(int parcaNo)` diye iki functional metoda ayır; akışı çağıran yönetsin.

5. Communicational cohesion (iletişimsel)

Tanım: Procedural ile aynı (*ürünün prosedürüne göre ilişkili eylemler*), **ek olarak bütün eylemler aynı veri üzerinde çalışır**. Kitap örnekleri: `update_record_in_database_and_write_it_to_audit_trail`, `calculate_new_coordinates_and_send_them_to_terminal`.

```
void KaydiGuncelleVeDenetimeYaz(Kayit k)   // iki is, ayni veri: k
{
    veritabani.Guncelle(k);
    denetimIzi.Yaz(k);
}
```

Neden kötü? Hâlâ birden fazla iş yapar; yeniden kullanılabilirlik düşüktür. Hoca'nın Lego benzetmesi: 5 birimlik Lego'yu sadece 5 birime ihtiyaç olduğunda kullanabilirsin; 1 birimlik Lego her yerde kullanılır. Modül tek iş yaptıkça yeniden kullanılabilirliği artar.

Düzeltme: `KaydiGuncelle(Kayit k)` ve `DenetimeYaz(Kayit k)` olarak ikiye böl.

6. Functional cohesion (işlevsel)

Tanım: Modül **tam olarak tek bir eylem** yapar. Kitap örnekleri: `get_temperature_of_furnace`, `compute_orbital_of_electron`, `write_to_diskette`, `calculate_sales_commission`.

```
double SatisKomisyonuHesapla(double satisTutari)
{
    return satisTutari * 0.05;
}
```

Avantajları: daha **yeniden kullanılabilir**; corrective bakım kolay (**hata izolasyonu**, daha az regression fault); ürünü **genişletmek kolay**.

7. Informational cohesion (bilgisel) — en iyi

Tanım: Modül bir dizi eylem yapar; **her eylemin kendi giriş noktası (entry point)** ve **bağımsız kodu** vardır ve **hepsi aynı veri yapısı üzerinde** çalışır. Bu, özünde bir **abstract data type**'tır (ADT); **sınıflar** informational cohesion örneğidir.

```
class IsKuyrugu                          // veri + o veri uzerindeki islemler
{
    private Queue<Is> kuyruk = new Queue<Is>();      // veri gizli
    public void Ekle(Is i)  { kuyruk.Enqueue(i); }   // giris noktasi 1
    public Is Cikar()       { return kuyruk.Dequeue(); } // giris noktasi 2
    public int Uzunluk()    { return kuyruk.Count; } // giris noktasi 3
}
```

Kitap örneği (Figure 7.6): `sales_region_table` verisi ile onun üzerinde çalışan `initialize_sales_region_table`, `update_sales_region_table`, `print_sales_region_table` işlemleri; her birinin kendi giriş ve çıkışı var.

4.3.1 Cohesion türünü bulma algoritması (sınavda koddan tür bulurken)

1. Modül **tek bir eylem** mi yapıyor? ⇒ **Functional**.
2. Birden çok eylem var ve her birinin **ayrı giriş noktası** var, hepsi **aynı (gizli) veri yapısı** üzerinde mi (sınıf/ADT)? ⇒ **Informational**.
3. Eylemler birbirinden **tamamen ilgisiz** mi? ⇒ **Coincidental**.
4. İlgili eylemlerden **biri, dışarıdan gelen bir kod/bayrakla (opCode, if/switch) seçilip** mi çalışıyor? ⇒ **Logical**.
5. Eylemler sadece **aynı zamanda yapıldıkları için** mi (başlangıç/bitiş)? ⇒ **Temporal**.
6. Eylemler ürünün **iş akışında art arda** gelen adımlar mı?
   - Hepsi **aynı veri** üzerinde ⇒ **Communicational**.
   - Farklı veriler üzerinde ⇒ **Procedural**.

Cevapta mutlaka gerekçe yaz

Sadece adı yazmayın: *"It has logical cohesion because the module performs a series of related actions and one of them is selected by the calling module through opCode."* Ardından düzeltilmiş kodu verin ve yeni seviyeyi söyleyin: *"After refactoring each method has functional cohesion."* İki seviye arasında kararsız kalırsanız (ör. coincidental mı temporal mı) hoca da derste *"coincidental ya da temporal"* diye iki cevabı kabul etti; gerekçenizi yazın.

4.3.2 Hocanın derste çözdüğü örnekler

Sıcaklık örnekleri (Hafta 5):

Modül	Cohesion	Gerekçe
compute average daily temperatures at various sites	Functional	tek iş: ortalama hesaplama
initialize sums and open files	Coincidental	ilgisiz işler (hoca; zamana bağlı görülürse temporal)
create new temperature record	Functional	tek iş
read in site, time, and temperature	Procedural	kitapta functional; hoca: "iki ayrı değişken okunuyor, procedural"
edit site, time, or temperature field	Logical	birden çok işlemden biri seçiliyor ("or")
store temperature record	Functional	tek iş
store record for specific site	Functional	tek iş
close files and print average temperatures	Coincidental / Temporal	ilgisiz ama bitişte art arda yapılıyor

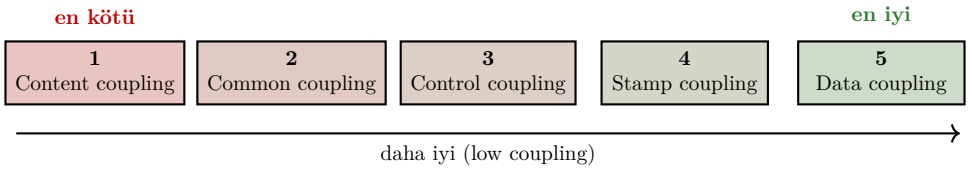
2025 dersinde tahtaya yazılan kod:

```
if (isDoc) UsePrinter("network");
else       UsePdfWriter(new int[] {1, 2, 3, 4, 5});

void UsePrinter(string b)  { Console.WriteLine(b); }                    // functional cohesion
void UsePdfWriter(int[] a) { Console.WriteLine(a[0] + a[a.Length-1]); } // functional
    cohesion
void WriteTimePrint()      { Console.WriteLine(DateTime.Now); Print(true); } // coincidental
    ya da temporal
```

Ek gözlem (coupling): `UsePdfWriter`'a bütün dizi gönderilip yalnızca ilk ve son eleman kullanılıyor ⇒ çağıran ile `UsePdfWriter` arasında **stamp coupling**. `Print(true)` bir bayrak (control element) gönderiyorsa **control coupling**.

4.4 Coupling (modüller arası bağlılık): 5 seviye



1. Content coupling (içerik) — en kötü

Tanım: Bir modül **diğerinin içeriğine doğrudan başvurur**. Örnekler: `p`, `q`'nun bir **komutunu değiştirir**; `p`, `q`'nun **yerel verisine** sayısal bir kayma (displacement) ile başvurur; `p`, `q`'nun içindeki bir **yerel etikete dallanır** (`goto`).

```
modul M2:  ...  BB:  x = x + 1;  ...   // M2'nin icindeki etiket
modul M1:  ...  goto BB;               // M1, M2'nin icine dallaniyor
```

Neden kötü? `q`'daki neredeyse her değişiklik (hatta `q`'yu yeni bir derleyiciyle derlemek) `p`'nin de değişmesini gerektirir; bir modüldeki hata doğrudan diğerini etkiler.

2. Common coupling (ortak)

Tanım: İki modül **aynı global veriye yazma erişimine** sahiptir. Örnekler: `cca` ve `ccb` aynı `global_variable`'ı okuyup değiştirir; iki modül aynı veritabanı kaydını okuyup yazabilir; FORTRAN/COBOL `common`.

```
static class Ortak { public static int durum = 0; }      // global veri

void Modul3() { if (Ortak.durum == 0) Islem1(); else Islem2(); }
void Modul4() { Ortak.durum = 1; }   // Modul3'un davranisini degistirir
```

Neden kötü? Yapısal programlama ruhuna aykırı, kod okunamaz (döngüyü ne bitiriyor?); **yan etkiler**: bir modülü anlamak için global değişkeni değiştiren bütün modülleri okumak gerekir; global değişkenin bildirimi değişince diğer modüller de değişmeli; yeniden kullanmak zor; **clandestine (gizli) common coupling**: `p`'yi hiç değiştirmeden `p` ile ürünün geri kalanı arasındaki bağ değişebilir (Linux çekirdeği örneği); modül gereğinden fazla veriye maruz kalır ⇒ **bilgisayar suçu** riski.

Düzeltme: global değişkeni kaldır; veriyi parametreyle gönder (data coupling) ya da veriyi ve onu değiştiren işlemleri bir sınıfa kapsülle (information hiding).

3. Control coupling (denetim)

Tanım: Bir modül diğerine bir **kontrol öğesi** (control element) gönderir: çağrılan modülün *ne yapacağını* çağıran belirler. Örnekler: **logical cohesion'a sahip bir modüle opCode göndermek**; argüman olarak bir **control switch** göndermek.

```
void M2() { M1(3); }                    // M2, M1'in hangi isi yapacagini secer
void M1(int a)                          // M1 icinde logical cohesion
{
    if (a == 1) Gorev1(); else if (a == 2) Gorev2(); else Gorev3();
}
```

Kitaptaki ayrım: `q`, `p`'ye "*başaramadım*" derse bu **veridir**; "*başaramadım, ABC123 hata mesajını yaz*" derse bu **kontroldür**.

Neden kötü? Modüller bağımsız değildir: çağıran, çağrılanın iç yapısını ve mantığını bilmek zorundadır; yeniden kullanımı azaltır. Genellikle **logical cohesion** ile birlikte görülür.

Düzeltme: her görevi ayrı metot yap; çağıran doğrudan `Gorev3()`'ü çağırsın (her metot functional, aradaki bağ data/yok).

4. Stamp coupling (damga / veri yapısı)

Tanım: Parametre olarak bir **veri yapısı** gönderilir ama çağrılan modül bu yapının **bazı (hepsi değil)** bileşenleri üzerinde çalışır. Örnek: `calculate_withholding(employee_record)`.

```
struct Ogrenci { public int Id; public string Ad, Soyad, KullaniciAdi, Sifre; }

void AdSoyadYaz(Ogrenci o)                  // tum kayit gonderiliyor
{
    Console.WriteLine(o.Ad + " " + o.Soyad); // ama sadece Ad ve Soyad kullaniliyor
}
```

Hocanın vurgusu: **sadece struct değil**: bir **dizi** gönderilip yalnızca bazı elemanları kullanılıyorsa da stamp coupling vardır.

Neden kötü? Kayıttaki hangi alanların kullanıldığı ya da değiştirildiği, modülün tamamı okunmadan anlaşılmaz; yeniden kullanılamaz; **gereğinden fazla veri** geçer: kontrolsüz veri erişimi **bilgisayar suçuna** yol açabilir (hoca: "kullanmadığı hâlde şifreyi gönderiyorsunuz, o modüle şifre yetkisi veriyorsunuz").

Düzeltme: sadece gereken alanları gönder: `AdSoyadYaz(string ad, string soyad)` ⇒ data coupling.

Yapı göndermek her zaman kötü değildir

Yapının **bütün** bileşenleri kullanılıyor veya değiştiriliyorsa sorun yoktur, bu **data coupling**'dir: `invert_matrix(original_matrix, inverted_matrix)`, `print_inventory_record(warehouse_record)`.

5. Data coupling (veri) — en iyi

Tanım: Bütün parametreler **homojen veri öğeleridir**: basit parametreler ya da **bütün elemanları kullanılan** veri yapıları. Örnekler: `display_time_of_arrival(flight_number)`, `compute_product(first_number, second_number)`, `get_job_with_highest_priority(job_queue)`.

```
double AlanHesapla(double en, double boy) { return en * boy; }
```

İçerik, ortak, kontrol ve damga bağlılığının zorlukları yoktur; **bakım kolaydır**.

4.4.1 Coupling türünü bulma algoritması

1. Bir modül diğerinin **kodunu/etiketini/yerel verisini** değiştiriyor ya da ona dallanıyor mu? ⇒ **Content**.
2. İki modül aynı **global** veriyi (static alan, global değişken, aynı DB kaydı) okuyup **yazıyor** mu? ⇒ **Common**.
3. Gönderilen parametre çağrılanın **hangi işi yapacağını belirleyen** bir kod/bayrak mı? ⇒ **Control**.
4. Bir struct/kayıt/dizi gönderiliyor ama **bir kısmı** mı kullanılıyor? ⇒ **Stamp**.
5. Gönderilen her şey basit veri ya da tamamen kullanılan yapı mı? ⇒ **Data**.

4.5 Sınav tipi örnek: kodu verip tür ve düzeltme isteme

2025–26 vizesine benzer örnek

Aşağıdaki metodun cohesion seviyesini bulun, gerekçesini yazın ve daha iyi bir cohesion/coupling ile yeniden yazın.

```
class Siparis
{
    public double toplamTutar;
    public void kdvekle_indirimuygula(int indirimOrani)
    {
        toplamTutar = toplamTutar - toplamTutar * indirimOrani / 100.0; // indirim
        toplamTutar = toplamTutar + toplamTutar * 0.20;                 // KDV
    }
}
```

Çözüm

Cohesion: Communicational. Metot iki ayrı eylem yapıyor (indirim uygulama ve KDV ekleme). Bu eylemler ürünün prosedürüne göre art arda gelen adımlardır (önce indirim, sonra KDV) **ve ikisi de aynı veri** (`toplamTutar`) üzerinde çalışıyor. Tek iş yapmadığı için functional değildir; seçim yapılmadığı için logical değildir.

English: The method has communicational cohesion because it performs two actions (applying the discount and adding VAT) that are related by the procedure of the product and both operate on the same data (toplamTutar).
Düzeltme: Her işi ayrı bir metoda böl (her biri **functional**); veriyi gizle ve işlemleri veriyle birlikte sınıfa koy (sınıf **informational cohesion**); parametreler basit veri (**data coupling**).

```
class Siparis
{
    private double toplamTutar;                         // information hiding
    public Siparis(double tutar) { toplamTutar = tutar; }

    public void IndirimUygula(int indirimOrani)       // functional
    { toplamTutar -= toplamTutar * indirimOrani / 100.0; }

    public void KdvEkle(double kdvOrani)              // functional
    { toplamTutar += toplamTutar * kdvOrani; }

    public double Tutar() { return toplamTutar; }
}
// kullanim:  s.IndirimUygula(10);  s.KdvEkle(0.20);
```

Not: Öğrencilerin bir kısmı "procedural", "logical" veya "temporal" yazdı. Kitaptaki tanıma göre ölçüt **aynı veri üzerinde çalışma** olduğundan doğru seviye *communicational*'dır; "sequential cohesion" Schach'ın 7 seviyesinde yoktur.

4.6 Data encapsulation, ADT ve information hiding

4.6.1 İş kuyruğu örneği

Büyük bir ana bilgisayar için işletim sistemi: yüksek, orta ve düşük öncelikli işler için **üç kuyruk**; iş gönderilince uygun kuyruğa eklenir, çalışmaya hazır olunca kuyruktan çıkarılıp bellek ayrılır.

- **Tasarım 1:** kuyruk işlemleri ürünün her yerine dağılmış ⇒ **düşük cohesion**.
- **Tasarım 2:** `m_encapsulation` modülü: `job_queue` veri yapısı ile `initialize_job_queue`, `add_job_to_queue`, `delete_job_from_queue` bir arada ⇒ **informational cohesion**. Bu, **data encapsulation**'dır: bir veri yapısı ve onun üzerinde yapılan işlemler.

Data encapsulation'ın avantajları:

- **Geliştirme: abstraction**'ın bir örneğidir. Önce ürünü üst düzey kavramlarla (iş kuyrukları ve işlemleri) tasarla, kuyruğun nasıl gerçekleştirildiğini önemseme; sonra alt düzey bileşenleri tasarla, onların nasıl kullanılacağını önemseme. Hoca: *abstraction = gereksiz ayrıntıyı gizleyip ilgili ayrıntıyı vurgulayarak stepwise refinement yapmak.*
- **Bakım:** değişmesi muhtemel yerleri belirle ve değişikliğin etkisini en aza indir. Veri yapıları pek değişmez, gerçekleştirme ayrıntıları değişebilir. Kuyruk iki yönlü bağlı listeye çevrilse bile onu kullanan modüller değişmez, sadece yeniden derlenir.

4.6.2 Abstract data type (ADT)

Tek bir kuyruk yetmez, üç kuyruk gerekir: **veri tipi + o tipin örnekleri üzerinde yapılan işlemler = abstract data type**. Hoca: *ADT, informational cohesion'a sahip veri tipidir; sınıflar ADT örneğidir.*

- **Data abstraction:** tasarımcı ADT düzeyinde düşünür.

- **Procedural abstraction:** bir prosedür tanımlamak = dili genişletmek.

4.6.3 Information hiding

Data abstraction ve procedural abstraction daha genel bir kavramın örnekleridir: **information hiding**. Modüller, **değişmesi muhtemel öğeler gizlenecek** şekilde tasarlanır; gelecekteki değişiklik **yerelleşir** ve diğer modülleri etkileyemez. C#/C++/Java'da `private` alanlar ve `public` metotlar/property'ler ile sağlanır (*en az yetki prensibi*). Public alanlar information hiding'i bozar.

4.6.4 Nesneler, kalıtım, polymorphism ve dynamic binding

- **Birinci inceltme:** ürün ADT'ler cinsinden tasarlanır; değişkenler ("nesneler") ADT'lerin örnekleridir.
- **İkinci inceltme: class** = kalıtımı destekleyen ADT; nesneler sınıfların örnekleridir.
- **Inheritance:** `HumanBeingClass` (age, height, gender); `ParentClass` onun alt sınıfı olup bütün özelliklerini miras alır ve kendininkileri ekler (nameOfOldestChild, numberOfChildren). Kalıtım nesne yönelimli dillerin (Smalltalk, C++, Ada 95, Java, C#) temel özelliğidir; klasik dillerde (C, COBOL, FORTRAN) yoktur. UML'de büyük **boş üçgen** ile gösterilir; aggregation **boş elmas**, association **düz çizgi**.
- **Polymorphism ve dynamic binding:** Klasik paradigmada dosya açmak için doğru fonksiyonu açıkça seçeriz (`open_disk_file`, `open_tape_file`...). Nesne yönelimli paradigmada `myFile.open()` yazılır; hangi `open` metodunun çalışacağı **çalışma zamanında** belirlenir (**dynamic binding**); `open` farklı sınıfların nesnelerine uygulanabilir (**polymorphic**). `checkOrder(b : Base)`, Base'in her alt sınıfının nesnesine uygulanabilir.
- Polymorphism ve dynamic binding hem güçlü hem zayıf yandır: belirli bir metodun birden çok olası sürümü varsa kodu anlamak zorlaşır, bakım olumsuz etkilenebilir.

4.6.5 Nesne yönelimli paradigmanın güçlü ve zayıf yanları

Başarısının nedenleri: veri ve işlemlere genel olarak eşit önem verir; iyi tasarlanmış bir nesne (high cohesion, low coupling) bir fiziksel varlığın bütün yönlerini modeller; gerçekleştirme ayrıntıları gizlidir.

Zayıf yanları: geliştirme eforu ve boyut büyük olabilir (ilk nesne yönelimli proje beklenenden büyük, özellikle GUI varsa; ama sınıflar sonraki projede yeniden kullanılabilir); kalıtım sorun çıkarabilir: **fragile base class problem** (taban sınıftaki değişiklik bütün alt sınıflara dalga gibi yayılır; bu yüzden sınıflar baştan dikkatle tasarlanmalı); alt sınıf ebeveynin bütün özelliklerini miras alır, ağacın altındaki nesneler şişebilir: *"Use inheritance where appropriate"*, gereksiz miras alınan özellikleri hariç tut; polymorphism ve dynamic binding sorun çıkarabilir; kötü nesne yönelimli kod yazmak özellikle kolaydır. Bir gün yerini daha iyisi alacaktır (ör. aspect-oriented programming).

4.7 SOLID ilkeleri (giriş)

Hoca cohesion/coupling'i anlatırken SOLID'i bağladı; ayrıntıları uygulama haftalarında (final konusu) işlenir, ama vizede kod düzeltirken gerekçe olarak yazmak işe yarar:

İlke	Anlamı	Cohesion/coupling ile bağı
Single Responsibility	Bir sınıfın/modülün değişmek için tek bir nedeni olmalı; tek iş.	Coincidental, logical, procedural ve communicational modülleri böl ⇒ functional.
Open-Closed	Genişletmeye açık, değişikliğe kapalı.	Logical cohesion'da yeni durum için `if` eklemek OCP ihlali; arayüz + polymorphism ile çöz.
Liskov Substitution	Alt sınıf, taban sınıfın yerine geçebilmeli, davranışını bozmamalı.	Kalıtımı "where appropriate" kullan.
Interface Segregation	Sınıflar kullanmadıkları metotları içeren büyük arayüzlere zorlanmamalı.	Stamp coupling'e benzer: gereğinden fazlasına bağımlı olma.
Dependency Inversion	Üst düzey modüller somut sınıflara değil soyutlamalara (interface) bağlı olmalı.	Coupling'i düşürür.

4.8 Kısa tanım soruları

Module A lexically contiguous sequence of program statements, bounded by boundary elements, with an aggregate identifier.

Cohesion The degree of interaction within a module. (Seven levels: coincidental, logical, temporal, procedural, communicational, functional, informational.)

Coupling The degree of interaction between two modules. (Five levels: content, common, control, stamp, data.)

Good design High cohesion and low coupling (maximal interaction within a module, minimal interaction between modules).

Data encapsulation A data structure together with the operations performed on that data structure.

Abstract data type A data type together with the operations performed on instantiations of that data type (a module with informational cohesion; a class).

Information hiding Designing modules so that items likely to change are hidden; future change is localized.

Class An abstract data type that supports inheritance.

Dynamic binding The method to be invoked is determined at run time according to the class of the object.

Polymorphism The same method can be applied to objects of different classes, each behaving in its own way.

Fragile base class problem A change to a base class ripples through all of its subclasses.

5 ISO/IEC 25010 Yazılım Ürün Kalitesi Modeli

! Hoca derste vurguladı Hafta 4 ve 2025 WhatsApp

"These characteristics I will ask in the exam ... please learn these characteristics, all of these characteristics in ISO 25000 series, these are very important." 2025 WhatsApp: *"ISO tablosunu sordum, hepsine açıklama yazmamızı ister misiniz dedim, hoca hayır dedi; her başlığın altında maddeler olduğu için hepsinden şu kadar tanesini yazın derim dedi." "İkişer başlık bilesiniz yeter dedi."*

2025–26 vizesi: ilk soru, 18 puan; **ana başlık 1'er puan, alt başlık 0,5'er puan**. $9 \times 1 + 18 \times 0{,}5 = 18$, yani her ana başlıktan **en az iki** alt başlık bekleniyor. Önceki yıllarda tablonun tamamı istendi (toplam 20–25 puan).

5.1 Model nedir?

ISO/IEC 25000 serisi (SQuaRE: *Systems and software Quality Requirements and Evaluation*) yazılım ürün kalitesini değerlendirme standartlarıdır. ISO/IEC 25010 bunun **kalite modelini** tanımlar: bir yazılım ürünü değerlendirilirken hangi kalite karakteristiklerinin dikkate alınacağını belirler. Bir sistemin kalitesi, paydaşlarının *belirtilmiş ve örtük ihtiyaçlarını* karşılama derecesidir.

Hocanın vurgusu: Bu karakteristikler *bir yazılımın ne yaptığını* değil, **dokümanlarda nelerin tanımlanması gerektiğini** söyler. Örneğin yanıt süresinin düşük ya da yüksek olması önemli değildir, **belgelerde tanımlanmış olması** önemlidir; tanımlanmışsa izlenebilir, ölçülebilir ve iyileştirilebilir. (Hoca güvenlik için ayrıca ISO 27000 serisinden söz etti.)

5.2 Hocanın tablosu: 9 karakteristik, 40 alt karakteristik (ISO/IEC 25010:2023)

2025'te derste ve WhatsApp'ta paylaşılan tablo (iso25000.com, *Software Product Quality*) budur. Ezber için:

Karakteristik	Alt karakteristikler	
1. Functional Suitability	Functional completeness, Functional correctness, Functional appropriateness	(3)
2. Performance Efficiency	Time behaviour, Resource utilization, Capacity	(3)
3. Compatibility	Co-existence, Interoperability	(2)
4. Interaction Capability	Appropriateness recognizability, Learnability, Operability, User error protection, User engagement, Inclusivity, User assistance, Self-descriptiveness	(8)
5. Reliability	Faultlessness, Availability, Fault tolerance, Recoverability	(4)
6. Security	Confidentiality, Integrity, Non-repudiation, Accountability, Authenticity, Resistance	(6)
7. Maintainability	Modularity, Reusability, Analysability, Modifiability, Testability	(5)
8. Flexibility	Adaptability, Scalability, Installability, Replaceability	(4)
9. Safety	Operational constraint, Risk identification, Fail safe, Hazard warning, Safe integration	(5)

Toplam: $3 + 3 + 2 + 8 + 4 + 6 + 5 + 4 + 5 = 40$ alt karakteristik.

Ezber ipuçları

- Ana başlıkların baş harfleri: **F**unctional suitability, **P**erformance, **C**ompatibility, **I**nteraction, **R**eliability, **S**ecurity, **M**aintainability, **F**lexibility, **S**afety ⇒ "**FP-CIR-SMFS**": "*Fonksiyonel Performans, Cihazlar İletişimde Rahat; Sistem Mutlaka Fazla Sağlam*".
- Functional suitability'nin üçü de "functional" ile başlar: *completeness, correctness, appropriateness* (tam mı, doğru mu, uygun mu).
- Performance: *zaman, kaynak, kapasite.*
- Compatibility: *birlikte var olma, birlikte çalışma* (co-existence, interoperability).

- Reliability: hata olmasın (faultlessness), erişilebilsin (availability), hataya rağmen çalışsın (fault tolerance), sonra toparlansın (recoverability).
- Security'nin ilk üçü klasik CIA'ya benzer: *Confidentiality, Integrity*; sonra *Non-repudiation, Accountability, Authenticity, Resistance* ("inkâr edemezsin, hesabını verirsin, kim olduğunu kanıtlarsın, saldırıya dayanırsın").
- Maintainability: Chapter 7 ile bağlantı: **Modularity** (iyi modülerleştirme), **Reusability**, **Analysability**, **Modifiability**, **Testability**.
- Flexibility: eski *portability*'nin genişlemiş hâli: *Adaptability, Scalability, Installability, Replaceability.*

5.3 Alt karakteristiklerin anlamları

Hoca açıklama istemiyor, ama anlamını bilmek maddeleri hatırlamayı kolaylaştırır.

Alt karakteristik	Anlamı
Functional Suitability — ürün belirtilmiş ve örtük ihtiyaçları karşılayan işlevleri sağlıyor mu?	
Functional completeness	İşlevler belirtilen bütün görevleri ve kullanıcı hedeflerini kapsıyor mu?
Functional correctness	Doğru sonuçları gereken hassasiyetle üretiyor mu?
Functional appropriateness	İşlevler görevlerin yerine getirilmesini kolaylaştırıyor mu?
Performance Efficiency — kullanılan kaynağa göre performans	
Time behaviour	Yanıt ve işlem süreleri, verim (throughput) gereksinimleri karşılıyor mu?
Resource utilization	Kullanılan kaynak (bellek, CPU) miktarı ve türü.
Capacity	Bir parametrenin üst sınırları (eşzamanlı kullanıcı, kayıt sayısı).
Compatibility — başka ürünlerle bilgi alışverişi / aynı ortamı paylaşma	
Co-existence	Ortak ortam ve kaynakları başka ürünlerle zarar vermeden paylaşabilme.
Interoperability	İki veya daha fazla sistemin bilgi alışverişi yapıp bu bilgiyi kullanabilmesi (hoca: e-Devlet ile kimlik doğrulama).
Interaction Capability (2011'de Usability) — kullanıcıyla etkileşim	
Appropriateness recognizability	Kullanıcı ürünün ihtiyacına uygun olup olmadığını anlayabiliyor mu?
Learnability	Kullanmayı öğrenme kolaylığı.
Operability	Kullanma ve kontrol etme kolaylığı.
User error protection	Kullanıcıyı hata yapmaktan koruma (hoca: öğrenci numarası kutusuna harf yazdırmamak; ilk düğmeye basılmadan ikinciyi pasif tutmak).
User engagement	Hoş, ilgi çekici, motive edici etkileşim (2011'de UI aesthetics).
Inclusivity	Farklı özellik ve yeteneklerdeki insanların kullanabilmesi (2011'de Accessibility).
User assistance	Kullanıcıya yardım, destek (yardım menüsü, ipuçları).
Self-descriptiveness	Ürünün kendini açıklaması; ekranın ne yapılacağını kendiliğinden göstermesi.
Reliability — belirtilen koşullarda belirtilen süre boyunca belirtilen işlevleri yerine getirme	
Faultlessness	Normal çalışmada hatasız işleme (2011'de Maturity).
Availability	Gerektiğinde çalışır ve erişilebilir olma.
Fault tolerance	Donanım/yazılım hatalarına rağmen amaçlandığı gibi çalışma (hoca: iki hata arası süre uzunsa güvenilirlik yüksek).
Recoverability	Kesinti/arıza sonrası veriyi kurtarıp istenen duruma dönebilme (kaç dakikada toparlanıyor?).
Security — bilgi ve veriyi yetki düzeyine uygun şekilde koruma	
Confidentiality	Veriye yalnızca yetkililerin erişmesi.
Integrity	Program veya verinin yetkisiz erişim/değişikliğe karşı korunması.
Non-repudiation	Eylem/olayların gerçekleştiğinin kanıtlanabilmesi, sonradan inkâr edilememesi.
Accountability	Bir varlığın eylemlerinin tekil olarak o varlığa izlenebilmesi.
Authenticity	Bir özne veya kaynağın kimliğinin iddia edildiği gibi olduğunun kanıtlanması.
Resistance	Saldırılara karşı direnç (2023'te eklendi).
Maintainability — iyileştirmek, düzeltmek, uyarlamak için değiştirilebilme	
Modularity	Bir bileşendeki değişikliğin diğerlerini en az etkileyecek şekilde ayrık bileşenlerden oluşma.
Reusability	Bir varlığın birden fazla sistemde kullanılabilmesi.
Analysability	Değişikliğin etkisini değerlendirme, hata nedenini bulma, değişecek parçayı belirleme kolaylığı.
Modifiability	Hata eklemeden, kaliteyi düşürmeden değiştirilebilme.
Testability	Test kriterleri konup testlerin yapılabilmesi.
Flexibility (2011'de Portability) — farklı/değişen ortamlara uyum	
Adaptability	Farklı veya gelişen donanım, yazılım, ortama uyarlanabilme.
Scalability	Artan/azalan iş yüküne göre ölçeklenebilme (2023'te eklendi).
Installability	Belirli ortamda başarıyla kurulabilme / kaldırılabilme.
Replaceability	Aynı ortamda aynı amaçla başka bir ürünün yerine geçebilme.
Safety (2023'te yeni) — insan hayatı, sağlık, mülk ve çevre için tehlike yaratmama	
Operational constraint	Güvenli çalışma sınırları içinde kalma.

Alt karakteristik	Anlamı
Risk identification	Riskleri belirleyebilme.
Fail safe	Arıza anında kendini güvenli moda alma.
Hazard warning	Tehlike durumunda uyarı verme.
Safe integration	Başka bileşenlerle güvenli bütünleşme.

5.4 2011 sürümü (8 karakteristik, 31 alt karakteristik)

Ders videolarında (2020–21) ve klasördeki `ISO25000.docx` notunda 2011 sürümü anlatılır. Eski test sorusu da buna göredir.

2011 karakteristiği	Alt karakteristikler
Functional Suitability	Functional completeness, Functional correctness, Functional appropriateness
Performance Efficiency	Time behaviour, Resource utilization, Capacity
Compatibility	Co-existence, Interoperability
Usability	Appropriateness recognizability, Learnability, Operability, User error protection, User interface aesthetics, Accessibility
Reliability	Maturity, Availability, Fault tolerance, Recoverability
Security	Confidentiality, Integrity, Non-repudiation, Accountability, Authenticity
Maintainability	Modularity, Reusability, Analysability, Modifiability, Testability
Portability	Adaptability, Installability, Replaceability

2011 → 2023 farkları: Usability → **Interaction Capability** (UI aesthetics → user engagement, accessibility → inclusivity; yeni: user assistance, self-descriptiveness); Maturity → **Faultlessness**; Security'ye **Resistance**; Portability → **Flexibility** (+ **Scalability**); yeni karakteristik **Safety**.

Hangi sürümü yazmalı?

Hoca 2025'te **2023 sürümünü** (9 başlıklı tablo) kullandı ve puanlamayı ona göre yaptı. Sınavda 9 ana başlığı ve her birinden en az 2–3 alt başlığı yazın. Hem 2011 hem 2023'te aynı kalan alt başlıkları (ör. Time behaviour, Co-existence, Confidentiality, Modularity, Installability) tercih etmek güvenlidir.

Çıkmış test sorusu

"Which one is NOT an ISO/IEC 25010 quality characteristic?" (reliability, portability, compatibility, **comprehensibility**, security). Cevap: **comprehensibility**.

5.5 Sınavda cevap şablonu

Ana başlık (1 p)	Alt başlıklar (0,5 p) — en az ikisi
Functional Suitability	completeness, correctness, appropriateness
Performance Efficiency	time behaviour, resource utilization, capacity
Compatibility	co-existence, interoperability
Interaction Capability	learnability, operability, user error protection, inclusivity
Reliability	faultlessness, availability, fault tolerance, recoverability
Security	confidentiality, integrity, non-repudiation, authenticity
Maintainability	modularity, reusability, analysability, modifiability, testability
Flexibility	adaptability, scalability, installability, replaceability
Safety	fail safe, hazard warning, risk identification

6 Gereksinim İş Akışı ve Use Case Diyagramları (Chapter 11)

! Hoca derste vurguladı Sınavdaki yeri

Eski vize notu: *"Use case (süreç) diagram, collaboration-sequence diagramlarını sınavda soruyorum."* 2024–25 vizesinde **ayrıntılı use case sorusu** geldi (alışveriş sitesi: sepete ekleme, arama, stok varsa satın alma) ve hoca vize sonrası çözümünü tahtada gösterdi. 2014 vizesinde *"Proje yöneticisi, lider, yazılımcıların olduğu bir proje takip sistemi: use case'lerini oluşturunuz"* soruldu. WhatsApp: *"geçen senelerde girenlerle konuştuğum kadarıyla brief description da istiyormuş, use case diyagramını çizmemizi de."*

6.1 Gereksinim iş akışının amacı

Sorulan soru: **"Ürün ne yapabilmeli?"** (*What must the product be able to do?*)

Yanılgı: "Müşterinin ne *istediğini* belirlemeliyiz." Doğrusu: müşterinin neye **ihtiyaç** duyduğunu belirlemeliyiz (*needs, not wants*). Kitaptaki meşhur cümle: *"I know you believe you understood what you think I said, but I am not sure you realize that what you heard is not what I meant!"*

- Bir sistem analistinin yazılımı ve işlevselliğini zihninde canlandırması zordur; müşteri için çok daha zordur. Bilgiyi müşteriden almak için **becerikli bir sistem analisti** gerekir; bu bilginin tek kaynağı müşteridir.
- Çözüm: müşteriden ilk bilgiyi al, bunu Unified Process'e girdi olarak kullan, Unified Process'in adımlarıyla müşterinin **gerçek ihtiyaçlarını** belirle.

6.2 Gereksinim iş akışının adımları

1. **Uygulama alanını (application domain) anla**: ürünün çalışacağı özel ortam.
2. **İş modelini (business model) kur**: müşterinin iş süreçlerini modelle.
3. İş modelini kullanarak **müşterinin gereksinimlerini** belirle.
4. Bu adımları **yinele** (iterate).

Requirements elicitation (capture): gereksinimleri keşfetme (görüşme, anket). **Requirements analysis:** ilk gereksinimleri inceltme ve genişletme.

6.2.1 Alanı anlamak ve sözlük (glossary)

Ekibin her üyesi uygulama alanını iyi tanımalıdır; **doğru terminoloji** şarttır (müşterinin dilini konuşamazsan gereksinimleri doğru çıkaramazsın). Bunun için bir **glossary** hazırlanır: alanda kullanılan teknik kelimelerin ve anlamlarının listesi.

6.2.2 İş modeli ve bilgi toplama teknikleri

Business model: bir kurumun iş süreçlerinin tanımı; müşterinin işini bir bütün olarak anlamayı sağlar (bilgisayarlaştırma konusunda müşteriye tavsiye vermek için şarttır).

- **Interviewing (görüşme)** — birincil tekniktir. Gereksinim ekibi müşteri ve kullanıcılarla görüşür.
  - İki tür soru: **close-ended** (belirli bir cevap gerektiren, kapalı uçlu) ve **open-ended** (görüşülen kişiyi konuşmaya teşvik eden, açık uçlu).

  - İki tür görüşme: **structured interview** (önceden planlanmış, çoğunlukla kapalı uçlu sorular) ve **unstructured interview** (sorular alınan cevaplara göre sorulur, çoğunlukla açık uçlu).
  - Görüşme kolay değildir: çok yapısız görüşme pek bilgi vermez; görüşmeci alanı çok iyi tanımalı ve her zaman açık fikirli olmalıdır. Görüşmeden sonra **yazılı rapor** hazırlanmalı ve bir kopyası görüşülen kişiye verilmelidir.

- **Questionnaire (anket):** yüzlerce kişinin görüşü gerekiyorsa.
- **İş formlarının incelenmesi:** müşterinin işini şu anda nasıl yaptığını gösterir.
- **Doğrudan gözlem:** çalışanları işlerini yaparken izlemek; modern hâli video kamera. Dezavantajı: kayıtları analiz etmek uzun sürer, çalışanlar mahremiyet ihlali olarak görebilir.

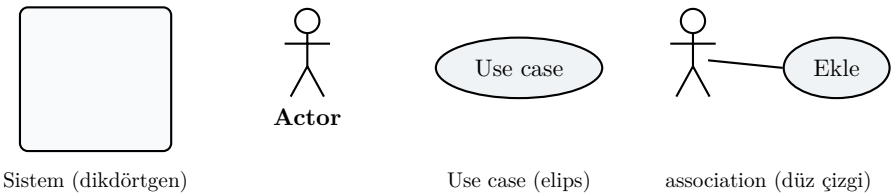
6.3 Use case ve aktör

Use case: yazılım ürününün kendisi ile kullanıcıları (**actors**) arasındaki bir etkileşimi modeller.

Actor: yazılım ürününün **dışındaki** dünyanın bir üyesidir. Genellikle ürünün kullanıcısıdır. Bir aktör ürüne göre bir **rol** oynar: kullanıcı olarak, başlatıcı (*initiator*) olarak veya use case'te kritik rol oynayan biri olarak.

- Bir kullanıcı birden fazla rol oynayabilir: bankanın müşterisi hem *Borrower* hem *Lender* olabilir.
- Bir aktör birden çok use case'e katılabilir: *Borrower*, *Borrow Money*, *Pay Interest on Loan*, *Repay Loan Principal* use case'lerinde aktördür. *Borrower* binlerce banka müşterisini temsil edebilir.
- **Aktör insan olmak zorunda değildir**: bir e-ticaret sistemi için kredi kartı şirketinin bilgi sistemi bir aktördür (ve tersi).
- Hoca: aktör **yazılımın çalıştığı yerde rol alan** kullanıcı/çalışandır; yazılımı geliştiren mühendisler aktör değildir. Aktörler bireyler değil, **rollerdir**.
- **Örtüşen aktörler** (overlapping actors) sorunu: hastane yazılımında bir use case'te *Nurse*, diğerinde *Medical Staff* aktörü. Daha iyisi: *Physician* ve *Nurse* aktörleri; ya da *Medical Staff* aktörü ve onun iki özelleşmesi *Physician* ve *Nurse* (**generalization**).

6.4 Use case diyagramının notasyonu



Sistem (dikdörtgen) | Use case (elips) | association (düz çizgi)

İlişki	Çizim	Anlamı ve okun yönü
Association	düz çizgi	Aktör ile use case arasında; aktör o use case'i başlatır/katılır.
«include»	kesikli ok + «include»	Ortak davranış ana use case'ten dışarı alınmıştır, **yeniden kullanım** içindir (istisna değildir). Ana use case çalışırsa included use case **kesinlikle** çalışır ("1 varsa 2 kesin gerçekleşir"). **Ok ana (base) use case'ten kullanılan (included) use case'e doğrudur.**
«extend»	kesikli ok + «extend»	**İstisnai veya nadir çalışan** durumlar (hata, iptal, zaman aşımı). Ana use case çalışsa bile extension çalışmayabilir ("1 var, 2 olmaya da bilir"). **Ok extension use case'ten genişletilen (base) use case'e doğrudur.** Bir istisna use case'i birden çok use case'i genişletebilir.
Generalization	düz çizgi + boş üçgen	Özel (child) aktör/use case'ten genel (parent) olana doğru; ok ucu **boş** ve **genele** bakar.

Ok yönleri ve include/extend ayrımı

- Hoca'nın notu: *"Kesikli ok üzerinde -include- yazar, okun yönü ana süreçten kullanılana doğrudur... Ana olaydan alt adımına doğru olacak, sınavda çıkıyor."*
- Generalization için: *"Okun ucu boş. Bu boş uç genelleşmiş aktöre doğrudur. Bunu unutmayalım, genele doğru okun yönü."*
- WhatsApp tartışması: *Payment* ile *Approve Card* arasında **include** vardır, çünkü kart onayı **her ödemede** yapılmalıdır; extend opsiyonel/istisnai durumlar içindir.

UML sunumundaki örnek (*PurchaseTicket*): *OutOfOrder, TimeOut, Cancel, NoChange* use case'leri «extends» ile *PurchaseTicket*'ı genişletir; *PurchaseSingleTicket* ve *PurchaseMultiCard* ortak *CollectMoney* davranışını «includes» ile kullanır.

6.5 Use case açıklaması (description)

Use case diyagramı sadece bir indekstir; modelin asıl "eti" use case açıklamalarıdır. Bütün use case'ler açıklanmalıdır. Hocanın kullandığı şablon:

Alan	İçerik
Use case name (unique name)	Benzersiz isim: *Delete Product*, *Kişi Ekle*
Brief description	Bir cümlelik özet
Participating actors	Katılan aktörler
Entry condition (pre-condition)	Use case başlamadan önce doğru olması gerekenler
Exit condition (post-condition)	Use case bitince doğru olanlar
Flow of events (step-by-step description)	Numaralı adımlar
Exceptional / special cases	İstisnai durumlar (genellikle «extend» use case'leri)

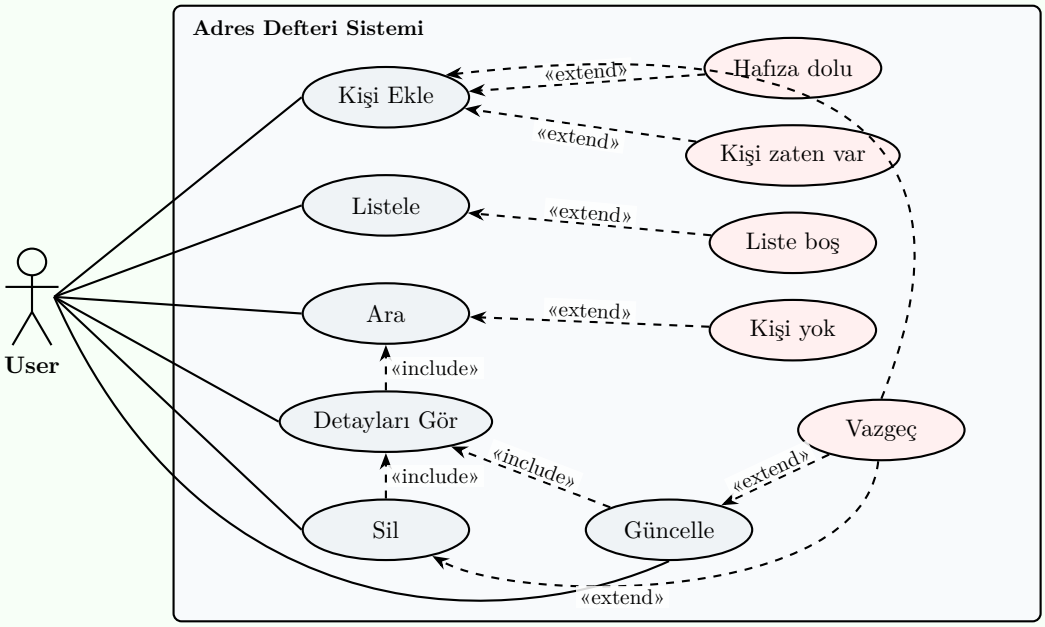
UML sunumundaki örnek: **Name:** Purchase ticket. **Participating actor:** Passenger. **Entry condition:** Passenger standing in front of ticket distributor; passenger has sufficient money. **Exit condition:** Passenger has ticket. **Event flow:** 1. Passenger selects the number of zones. 2. Distributor displays the amount due. 3. Passenger inserts money (at least the amount due). 4. Distributor returns change. 5. Distributor issues ticket. **Eksik olan?** *Exceptional cases!*

6.6 Hocanın çözdüğü örnek 1: Adres defteri

Ders örneği (2024–25 ve 2025–26) ve eski sınav senaryosu

Kişilere ait ad, soyad, adres ve telefon bilgisinin tutulduğu bir adres defteri tanımlanmak isteniyor. Adres defterine kişi ekleme, eklenmiş kişide güncelleme yapabilme, silme ve bulma (arama) işlemleri olacaktır. Ayrıca kişileri alfabetik olarak listeleyen bir işlev de gereklidir. Use case diyagramını çizip use case'leri açıklayınız.

Çözüm



Okuma: *Kişi Ekle* her zaman bilgi girişi ister; hafıza doluysa, kişi zaten varsa ya da kullanıcı vazgeçerse istisnai akışlar (**extend**) devreye girer. *Sil* ve *Güncelle* için önce kişinin detayları görülmeli, detay görmek için de önce arama yapılmalı (**include**, her seferinde yapılır).

Kişi Ekle (hocanın tahtadaki açıklaması):

- **Use-case name:** Kişi Ekle (Add Contact)
- **Brief description:** Kullanıcıdan alınan kişi bilgilerinin sisteme kaydedilmesi.
- **Participating actor:** User
- **Pre (entry) condition:** Eklenecek kişinin isim ve soyisim bilgileri bilinmelidir.
- **Post (exit) condition:** Kişi sisteme eklenir.
- **Step-by-step description:** 1. Kişinin isim ve soyismi alınır; varsa adres ve telefon bilgileri alınır. 2. Sisteme kaydedilir.
- **Special cases:** 1. Kişi zaten var. 2. Kullanıcı vazgeçmiş. 3. Hafıza dolu.

Kişi Güncelle:

- **Brief description:** Bulunan bir kişinin telefon ve adres bilgisinin güncellenmesi.
- **Entry condition:** Kişinin isim ve soyisim bilgilerinin belirtilmesi.
- **Exit condition:** Adres ve telefon bilgilerinin sistemde güncellenmesi.
- **Step-by-step:** 1. Güncellenecek kişinin isim ve soyismi girilir. 2. Sistemde bu kişi aranır. 3. Var ise yeni adres ve telefon bilgisi girilir. 4. Sistem günceller.

- **Exceptional cases:** Kişi yok; kullanıcı vazgeçti.

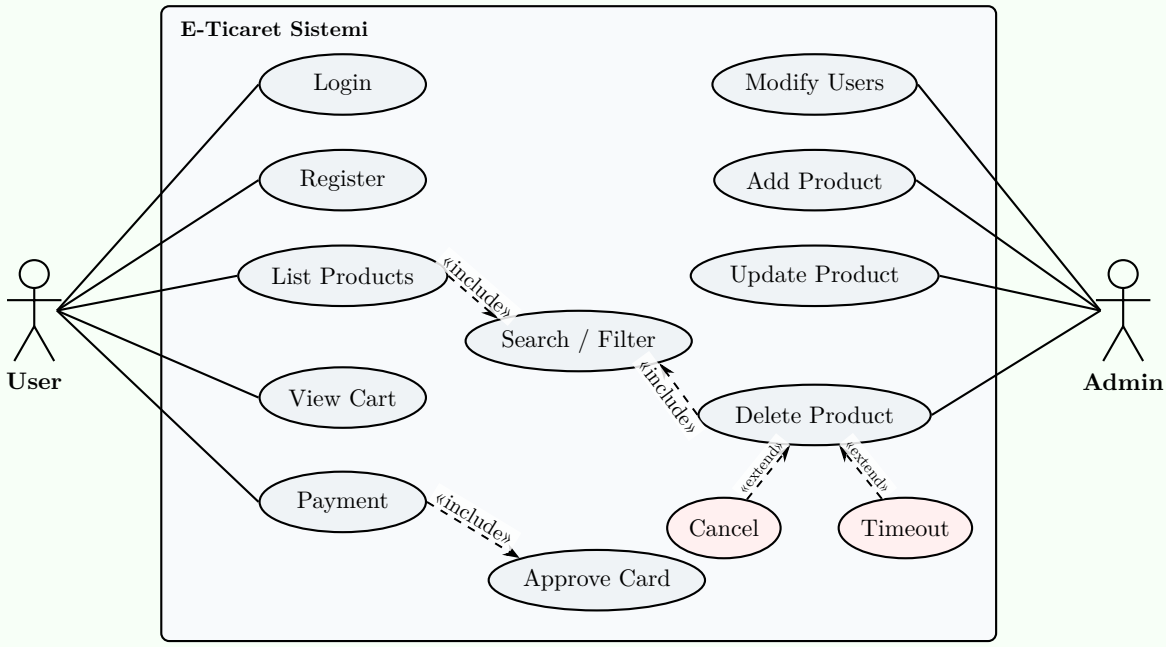
Listele: brief: kişilerin alfabetik sırayla listelenmesi; entry: defterde en az bir kişi olmalı; exit: kişiler ekranda alfabetik sırada; steps: 1. kullanıcı listele seçeneğini seçer, 2. sistem kişileri soyada göre sıralar, 3. listeyi gösterir; special: liste boş.

6.7 Hocanın çözdüğü örnek 2: E-ticaret sitesi (2024–25 vize sorusu)

2024–25 Vize (öğrenci aktarımı)

Bir alışveriş sitesi: kullanıcı giriş yapar / kayıt olur, ürünleri listeler ve arar/filtreler, sepete ekler, stokta varsa ödeme yaparak satın alır. Yönetici kullanıcıları ve ürünleri yönetir. Use case diyagramını çizip use case'leri ayrıntılı açıklayınız.

Çözüm



Delete Product (hocanın açıklaması):

- **Unique name:** Delete Product
- **Brief description:** Admin product deletion process is described.
- **Participating actor:** Admin
- **Pre-condition:** The product is found.
- **Post-condition:** The product is deleted.
- **Step-by-step description:** 1. Admin searches the product in the system. 2. The product details are shown. 3. Admin approves the deletion. 4. The product is deleted in 5 ms.
- **Special cases:** 1. Time out. 2. Cancel.

Payment:

- **Brief description:** The user pays for the products in the cart.
- **Participating actors:** User (and the bank / credit card system as a non-human actor, if modeled)

- **Pre-condition:** The user is logged in, the cart is not empty and the products are in stock.
- **Post-condition:** The payment is completed, the order is created and the stock is decreased.
- **Steps:** 1. User views the cart and selects payment. 2. User enters card information. 3. System approves the card (Approve Card, include). 4. System creates the order and shows a confirmation.
- **Special cases:** card is not approved; product is out of stock; user cancels.

Dikkat edilecekler: (1) Aktörler sistem dikdörtgeninin **dışında**, use case'ler **içinde**. (2) Aktör–use case arası **düz çizgi**. (3) Her include/extend okunun üzerine stereotip yazılır, yönler doğru çizilir. (4) Hoca kısıtı step-by-step'e somut yazmayı seviyor ("in 5 ms").

6.8 İlk gereksinimler: functional ve nonfunctional

İlk gereksinimler ilk iş modeline dayanır, sonra inceltilir. Gereksinimler **dinamiktir**: sık değişir; olası gereksinimlerin listesi ve müşterinin onayladığı use case'ler tutulur.

- **Functional requirement:** ürünün yapabilmesi gereken bir **eylemi** belirtir; çoğunlukla girdi ve çıktılarla ifade edilir. Requirements ve analysis iş akışlarında ele alınır.
- **Nonfunctional requirement:** ürünün **kendisinin özelliklerini** belirtir: platform kısıtları, yanıt süreleri, güvenilirlik. Bazıları ayrıntılı bilgi olmadığı için **design** iş akışına kadar bekler. (Hoca: *ne* yapacağından çok *nasıl* yapacağı; kaliteyle ilgili standartlar.)

6.9 MSG Foundation vaka çalışması

6.9.1 Alan bilgisi

Martha Stockton Greengage (MSG) Foundation genç çiftlere **düşük maliyetli ipotek (mortgage)** kredisi verir. Mütevelliler bir **pilot proje** ister: her hafta ev satın almak için ne kadar para olduğunu belirleyen bir yazılım. (Ön bilgi: ipotek, gayrimenkulün teminat olduğu kredidir; aylık ödeme *P & I* — principal and interest; sigorta ve emlak vergisi *escrow* hesabında biriktirilir; toplam aylık ödeme gelirin %28'inden az olmalıdır; *points* ve kapanış maliyetleri.)

6.9.2 İlk iş modeli

Her haftanın başında MSG o hafta ipoteğe ne kadar para ayırabileceğini tahmin eder. Çiftler her zaman başvurabilir; personel çiftin uygun olup olmadığına ve fon olup olmadığına bakar; uygunsa ipotek verilir ve haftalık geri ödeme MSG kurallarına göre hesaplanır. Use case'ler: *Estimate Funds Available for Week*, *Apply for an MSG Mortgage*, *Compute Weekly Repayment Amount*, *Manage an Investment*.

- **Neden *Applicants* bir aktör?** Başvuranlar yazılımla doğrudan etkileşmez; cevaplarını MSG personeli girer. Ama use case'i **başlatan** ve veriyi **sağlayan** onlardır; MSG personeli sadece aracıdır. Aynı mantıkla *Borrowers* da aktördür.

6.9.3 Haftalık fon formülü

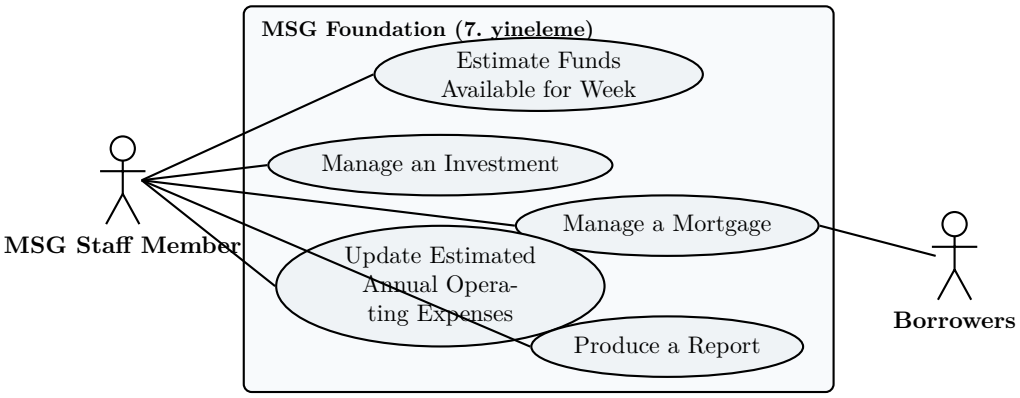
(1) Yatırımlardan tahmini yıllık gelir / 52; (2) tahmini yıllık işletme gideri / 52; (3) o haftanın tahmini ipotek ödemeleri toplamı; (4) o haftanın tahmini hibe (grant) toplamı; (5) haftanın başındaki kullanılabilir para = (1) − (2) + (3) − (4); (6) ev fiyatı (5)'ten büyük değilse ev için fon verilir; (7)

hafta sonunda harcanmayan para yatırıma gider. Pilot projede yalnızca üç tür veri gerekir: **investment, operating expenses, mortgage** verisi; üç rapor: haftalık hesap sonucu, bütün yatırımların listesi, bütün ipoteklerin listesi.

6.9.4 Yinelemelerle gereksinimlerin revizyonu

Use case diyagramı yedi yinelemede olgunlaşır:

1. *Estimate Investment Income for Week* eklenir: *Estimate Funds Available for Week*'in parçasıdır ⇒ «include».
2. *Update Estimated Annual Operating Expenses* ve *Estimate Operating Expenses for Week* eklenir.
3. *Compute Weekly Repayment Amount* ikiye bölünür: *Estimate Payments and Grants for Week* ve *Update Borrowers' Weekly Income*.
4. Unutulan *Manage a Mortgage* eklenir (ipotek ekleme/değiştirme/silme).
5. Unutulan *Produce a Report* eklenir (üç rapor).
6. *Update Borrowers' Weekly Income* silinir: haftalık gelir ipoteğin bir özelliğidir, *Manage a Mortgage* zaten bunu yapar. Bu bir **decrement**tir; hata yapınca silme sık olur. Önce mevcut yinelemede düzeltmeye çalışılır, olmazsa önceki yinelemeye geri dönülür.
7. İçerilen üç use case yalnızca tek bir use case'in parçası olduğu için *Estimate Funds Available for Week*'in içine alınır.



Yedinci yinelemede ulaşılan diyagram: bütün use case'leri MSG personeli yürütür; *Manage a Mortgage* use case'ini (haftalık gelir güncellemesi) *Borrowers* başlatır. (Chapter 13'te *Manage a Mortgage* ve *Manage an Investment* birleşip *Manage an Asset* olur.)

Doğru ve yanlış «include»

Bir use case **bir aktörle etkileşmek zorundadır**. *Estimate Payments and Grants for Week* aktörle etkileşmediği için kendi başına use case olamaz; *Estimate Funds Available for Week*'in parçasıdır (include ile gösterilir). Include'un olağan nedeni: **bir use case'in iki veya daha fazla use case'in parçası olması** (ör. ABD vergi formlarında aynı hesaplamanın üç kez yazılmaması).

6.10 Diğer konular

- **Test workflow:** gereksinimler gözden geçirilir; unutulan use case'ler ve tekrarlar bulunur (MSG'de *Manage a Mortgage* ve *Produce a Report* böyle bulundu).

- **Rapid prototyping** (klasik gereksinim fazı): müşterinin deneyebileceği hızlı bir model; hızlı yapılıp atılır. Nesne yönelimli yaklaşımda use case ve senaryolar her yinelemede inceltilir, prototip ise atılır; yine de kullanıcı arayüzünün prototipi (örnek ekran ve raporlar) gerekir. **Human factors:** kullanıcı dostu arayüz, menü, tutarlılık.
- **CASE araçları:** UML diyagramlarını çizmek ve kolay değiştirmek için grafik araçlar (IBM Rational Rose, Together, ArgoUML, System Architect).
- **Metrikler:** *volatility* ve *speed of convergence* (müşteri ihtiyaçlarının ne hızla belirlendiği); sonraki fazlarda yapılan değişiklik sayısı (geliştiriciden geliyorsa süreç kusurlu olabilir; müşteriden geliyorsa moving target).
- **Zorluklar:** müşteri kurumun çalışanları bilgisayarlaşmadan tehdit hissedebilir; ekip **müzakere** edebilmeli (ihtiyaçlar küçültülebilir); kilit çalışanların vakti olmayabilir; esneklik ve nesnellik şarttır.

6.11 Kısa tanım soruları

Use case Models an interaction between the software product itself and the users of that software product (actors).

Actor A member of the world outside the software product; a role (user, initiator or someone who plays a critical part). It need not be a human.

Business model A description of the business processes of an organization.

Glossary A list of technical words used in the domain, and their meanings.

Structured / unstructured interview Pre-planned, frequently close-ended questions / questions posed in response to the answers received, frequently open-ended.

«include» Behavior factored out of a use case for reuse; the base use case always performs it. Arrow from the base use case to the included use case.

«extend» Exceptional or seldom invoked cases factored out of the main flow. Arrow from the extending use case to the extended (base) use case.

Functional / nonfunctional requirement An action the product must perform / a property of the product itself (platform, response time, reliability).

7 Nesne Yönelimli Analiz (Chapter 13)

! Hoca derste vurguladı Sınavdaki yeri

2014 vizesi: *"Class yapısını oluşturun. Class tiplerini belirtin (Entity, Boundary, Control class) ... Collaboration veya sequence diyagramlarından birini seçip diyagram oluşturunuz."* 2025–26 vizesinin son sorusu (24 puan): rent-a-car otomasyonunun **sequence diyagramı**. WhatsApp: *"Sequence'de oklar üzerindeki çağrılar okun ucundaki sınıfın bir metodu olmalıdır."*

7.1 Analiz iş akışı ve üç sınıf türü

Object-oriented analysis (OOA) nesne yönelimli paradigmanın yarı biçimsel (semiformal) analiz tekniğidir (60'tan fazla eşdeğer teknik var; bugün tek geçerli alternatif Unified Process'tir). Bu iş akışında **sınıflar çıkarılır**. Analiz iş akışının iki amacı: gereksinimleri **daha derin anlamak** ve onları **bakımı kolay bir tasarım ve gerçekleştirmeye** götürecek şekilde tanımlamak.



Entity class



Boundary class



Control class

Tür	Ne modeller?	Örnek
Entity class	**Uzun ömürlü (long-lived) bilgiyi**; gerçek sistemdeki varlıkları.	Account Class, Investment Class, Mortgage Class, Kişi, Araç
Boundary class	Ürün ile **çevresi arasındaki etkileşimi**; genellikle **giriş/çıkış** ile ilgilidir. Her giriş ekranı, çıkış ekranı ve rapor kendi boundary sınıfıyla modellenir.	User Interface Class, Investments Report Class, Mortgages Report Class
Control class	**Karmaşık hesaplamaları ve algoritmaları**. Her önemsiz olmayan hesaplama bir control sınıfıyla modellenir.	Estimate Funds for Week Class, Elevator Controller

Bu üç tür UML'in **stereotype** genişletmesidir (`«entity»`, `«boundary»`, `«control»`). Hoca tahtada: entity = altı çizili daire, boundary = sol tarafında dikey çizgi olan daire, control = üstünde ok olan daire.

7.2 Entity sınıflarını çıkarmanın üç adımı

Aşağıdaki üç adım **artırımlı ve yinelemeli** yapılır:

1. **Functional modeling (işlevsel modelleme):** bütün use case'lerin **senaryolarını** sun. (*Senaryo, bir use case'in bir örneğidir.*)
2. **Entity class modeling (sınıf modelleme):** entity sınıflarını ve niteliklerini (attributes) belirle; aralarındaki ilişkileri ve etkileşimleri belirle; bunu bir **class diagram** olarak sun.
3. **Dynamic modeling (dinamik modelleme):** her entity sınıfı tarafından veya ona yapılan işlemleri belirle; bunu bir **statechart** olarak sun.

7.3 Asansör problemi (elevator problem)

Bir binada m kat ve n asansör var. (1) Her asansörde her kat için bir tane, toplam m düğme vardır; basılınca yanar ve asansörü o kata götürür; asansör o kata gelince söner. (2) İlk ve en üst kat hariç her katta iki düğme vardır (yukarı / aşağı); basılınca yanar; asansör o kata gelip istenen yönde hareket edince söner. (3) İsteği olmayan asansör, kapıları kapalı olarak bulunduğu katta bekler.

7.3.1 Functional modeling

Use case, ürün ile aktörler arasındaki etkileşimi **genel olarak** anlatır. Asansör probleminde sadece iki use case vardır: **Press an Elevator Button** ve **Press a Floor Button** (aktör: User). Senaryo bir use case'in bir örneğidir; hedef ürünü kavramak için **yeterli sayıda** senaryo incelenmelidir (hoca: ne çok fazla ne çok az).

- **Normal senaryo:** User A 3. katta yukarı düğmesine basar, 7. kata gitmek ister; düğme yanar; asansör gelir (içinde 1. katta binip 9. kat düğmesine basmış User B vardır); yukarı düğmesi söner, kapılar açılır, User A biner, 7. kat düğmesine basar...
- **Exceptional senaryo:** User A yukarı düğmesine basar ama asansör gelince *aşağı* gitmek istediğini fark eder ve yanlış asansöre biner vb.

7.3.2 Entity class modeling: isim çıkarma (noun extraction)

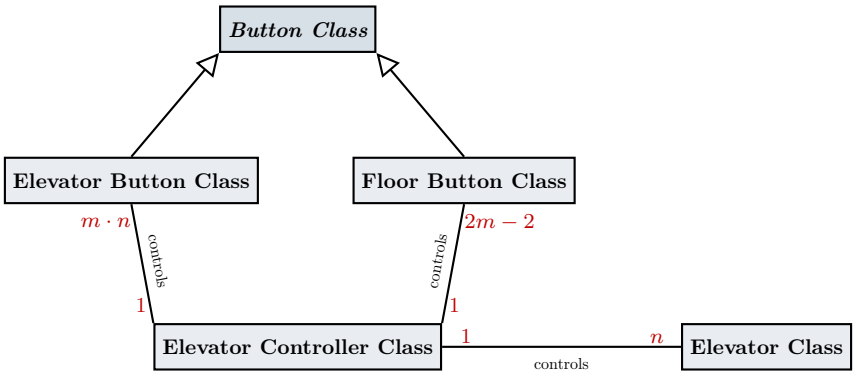
Sınıfları çıkarmanın yolları: senaryolardan çıkarmak (çok senaryo ⇒ çok aday sınıf tehlikesi), **CRC kartları** (alan bilgisi varsa), **noun extraction**. İsim çıkarma iki aşamalıdır:

1. **Kısa problem tanımı:** ürünü **tek paragrafta** tanımla.

   Buttons in elevators and on the floors control the movement of n elevators in a building with m floors. Buttons illuminate when pressed to request the elevator to stop at a specific floor; the illumination is canceled when the request has been satisfied. When an elevator has no requests, it remains at its current floor with its doors closed.

2. **İsimleri belirle** ve aday sınıf olarak kullan:
   - İsimler: *button, elevator, floor, movement, building, illumination, request, door.*
   - *floor, building, door* problem sınırının **dışında** ⇒ çıkar.
   - *movement, illumination, request* **soyut isimler** ⇒ çıkar (nitelik olabilirler).
   - Aday sınıflar: **Elevator Class** ve **Button Class**; alt sınıflar: **Elevator Button Class** ve **Floor Button Class**.

Sorun: düğmeler asansörlerle doğrudan iletişim kurmaz ⇒ ek bir sınıf gerekir: **Elevator Controller Class**. Böylece bütün ilişkiler **1-to-n** olur; bu tasarımı ve gerçekleştirmeyi kolaylaştırır.



7.3.3 CRC kartları

1989'dan beri OOA'da kullanılır. Her sınıf için bir kart doldurulur:

CLASS: sınıfın adı
RESPONSIBILITY: sınıfın işlevleri (ne yapar, neyi bilir)
COLLABORATION: çağırdığı / birlikte çalıştığı sınıflar

Bugün CRC kartları CASE araçlarıyla otomatik üretilir. **Güçlü yanı:** ekip üyeleri kartları canlandırınca (act out) **eksik veya hatalı öğeleri** ortaya çıkarmada çok güçlüdür (aynı zamanda iyi bir **test** tekniğidir). **Zayıf yanı:** entity sınıflarını bulmak için kullanılırsa **alan uzmanlığı** gerekir. (WhatsApp: CRC kartlarında sınıflar oklarla birbirine bağlanmaz.)

7.3.4 Dynamic modeling: statechart

Her sınıf için bir **UML statechart** çizilir. Durum (state), olay (event) ve koşul (predicate) diyagrama dağıtılmıştır. Statechart, senaryoların olaylarını modelleyerek kurulur. Notasyon: **dolu siyah daire** = başlangıç durumu; **içinde küçük siyah daire olan daire** = bitiş durumu; **köşeleri yuvarlatılmış dikdörtgen** = durum; **ok** = durumdan duruma geçiş (üzerinde onu tetikleyen olay).

7.3.5 Test workflow: CRC ile hataların bulunması

Elevator Controller'ın CRC kartında bir sorumluluk: *"1. Turn on elevator button"*. Bu nesne yönelimli paradigmaya **tamamen uygunsuzdur**: **responsibility-driven design** ve **information hiding** göz ardı edilmiştir. Doğrusu: *"1. Send message to Elevator Button Class to turn itself on."* Ayrıca bir sınıf unutulmuştur: asansör **kapılarının** çalışma sırasında değişen bir durumu vardır ⇒ **Elevator Doors Class** eklenir (güvenlik açısından da önemli). Class diagram değişince use-case diagram (değişmez), statechart'lar ve senaryolar yeniden gözden geçirilir.

7.3.6 God class ve dağıtık mimari

Üçüncü yinelemede *Elevator Controller Class* her şeyi yönetir. Bu bir **God class**'tır: çok fazla bilgiye maruz kalan ve çok fazla kontrole sahip sınıf; iyi bilinen bir **antipattern**'dir. Çözüm: **kontrolü dağıtmak (distributed decentralized architecture)**: her asansörün kendi *Elevator Subcontroller*'ı, her katın kendi *Floor Subcontroller*'ı vardır; $m+n$ alt kontrolcü istekleri işleyen bir *Scheduler* ile iletişim kurar; her katın üstünde ve altında bir *Sensor* asansör yaklaşınca/uzaklaşınca ilgili alt kontrolcüyü bilgilendirir. (Hoca: *tek bir sınıfa sorumluluk yüklemek yerine sorumlulukları dağıtmak.*) Analiz "şimdilik" doğrudur; tasarımda tekrar dönülebilir.

7.4 Boundary ve control sınıflarını çıkarmak

Boundary sınıflarını bulmak genellikle kolaydır: **her giriş ekranı, çıkış ekranı ve basılı rapor** bir boundary sınıfıdır. Her **hesaplama** genellikle bir control sınıfıdır.

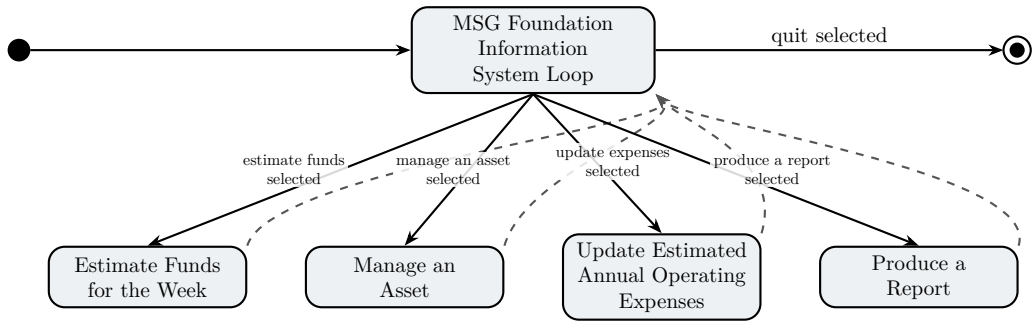
7.5 MSG Foundation vaka çalışması

7.5.1 İlk sınıf diyagramı (noun extraction)

Tek paragraf: *Weekly reports are to be printed showing how much money is available for mortgages. In addition, lists of investments and mortgages must be printed on demand.* İsimler: *report, money, mortgage, list, investment. report* ve *list* uzun ömürlü değil (report bir boundary sınıfı olacak); *money* soyut isim. Aday entity sınıfları: **Mortgage Class** ve **Investment Class**. İkisine yapılan işlemler çok benzer (ekleme, silme, değiştirme, istenince yazdırma) ⇒ ortak üst sınıf **Asset Class**; *Manage a Mortgage* ve *Manage an Investment* birleşip **Manage an Asset** olur. Durum diyagramı kontrol

edilince yıllık işletme giderlerinin saklanacağı bir sınıf olmadığı görülür ⇒ **MSG Application Class** eklenir ("iterate and increment" bazen *decrement* demektir).

7.5.2 İlk dinamik model



Dolu siyah daire başlangıç, içi noktalı daire bitiş durumudur. MSG personeli menüden beş komuttan birini seçer; her seçim bir **olaydır** ve bir durum geçişine neden olur. İşlem bitince sistem döngü durumuna geri döner.

7.5.3 Boundary ve control sınıfları

- Dört use case (Estimate Funds Available for Week, Manage an Asset, Update Estimated Annual Operating Expenses, Produce a Report) için tek ekran yeter ⇒ **User Interface Class**.
- Üç rapor ⇒ üç boundary sınıfı: **Estimated Funds Report Class**, **Mortgages Report Class**, **Investments Report Class**.
- Tek hesaplama ⇒ tek control sınıfı: **Estimate Funds for Week Class**.

7.6 Use-case realization

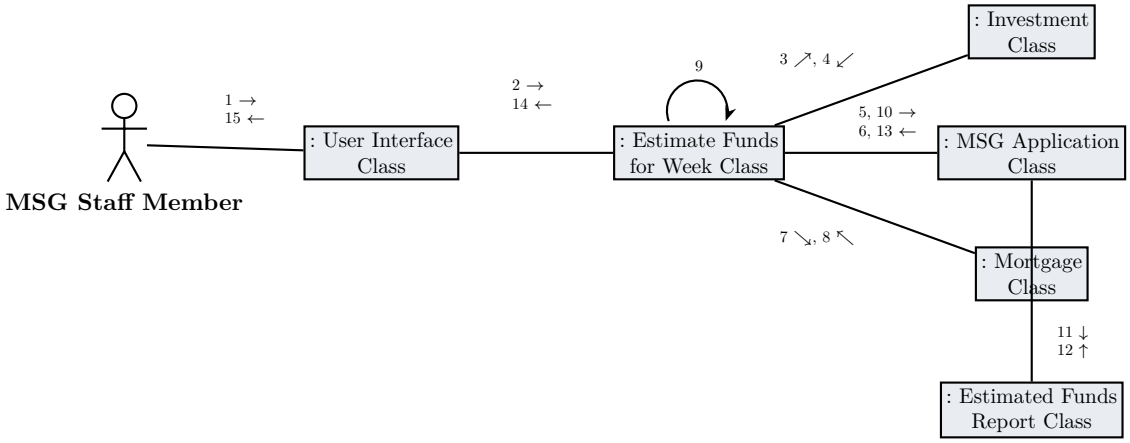
Use case'leri genişletip inceltme sürecine **use-case realization** denir (*realize* burada "gerçekleştirmek, başarmak" anlamındadır). Bir use case'in belirli bir senaryosunun gerçekleştirilmesi bir **interaction diagram** ile gösterilir: ya **sequence diagram** ya da **collaboration (communication) diagram**. Çalışan bir sistem sınıfları değil **nesneleri** kullanır: belirli bir ipotek `: Mortgage Class` biçiminde gösterilir.

Estimate Funds Available for Week use case'i için birbirinden türeyen artifact zinciri:

1. **Use case** (diyagram): aktör ile ürün arasındaki bütün olası etkileşimler.
2. **Use case açıklaması**: use case'in yazılı anlatımı.
3. **Class diagram**: use case'i gerçekleştiren sınıflar ve ilişkileri (nesneleri ve mesaj sırasını göstermez).
4. **Scenario**: use case'in belirli bir örneği.
5. **Collaboration diagram**: o senaryonun gerçekleştirilmesinde nesneler ve **gönderilme sırasına göre numaralanmış** mesajlar.
6. **Flow of events**: collaboration diyagramının yazılı anlatımı (müşteri anlamadan spesifikasyonu onaylamaz).
7. **Sequence diagram**: collaboration diyagramıyla **tamamen eşdeğer**.

Bu use case'e giren altı sınıf: *User Interface Class* (boundary), *Estimate Funds for Week Class* (control), *Mortgage Class*, *Investment Class*, *MSG Application Class* (entity), *Estimated Funds Report Class* (boundary).

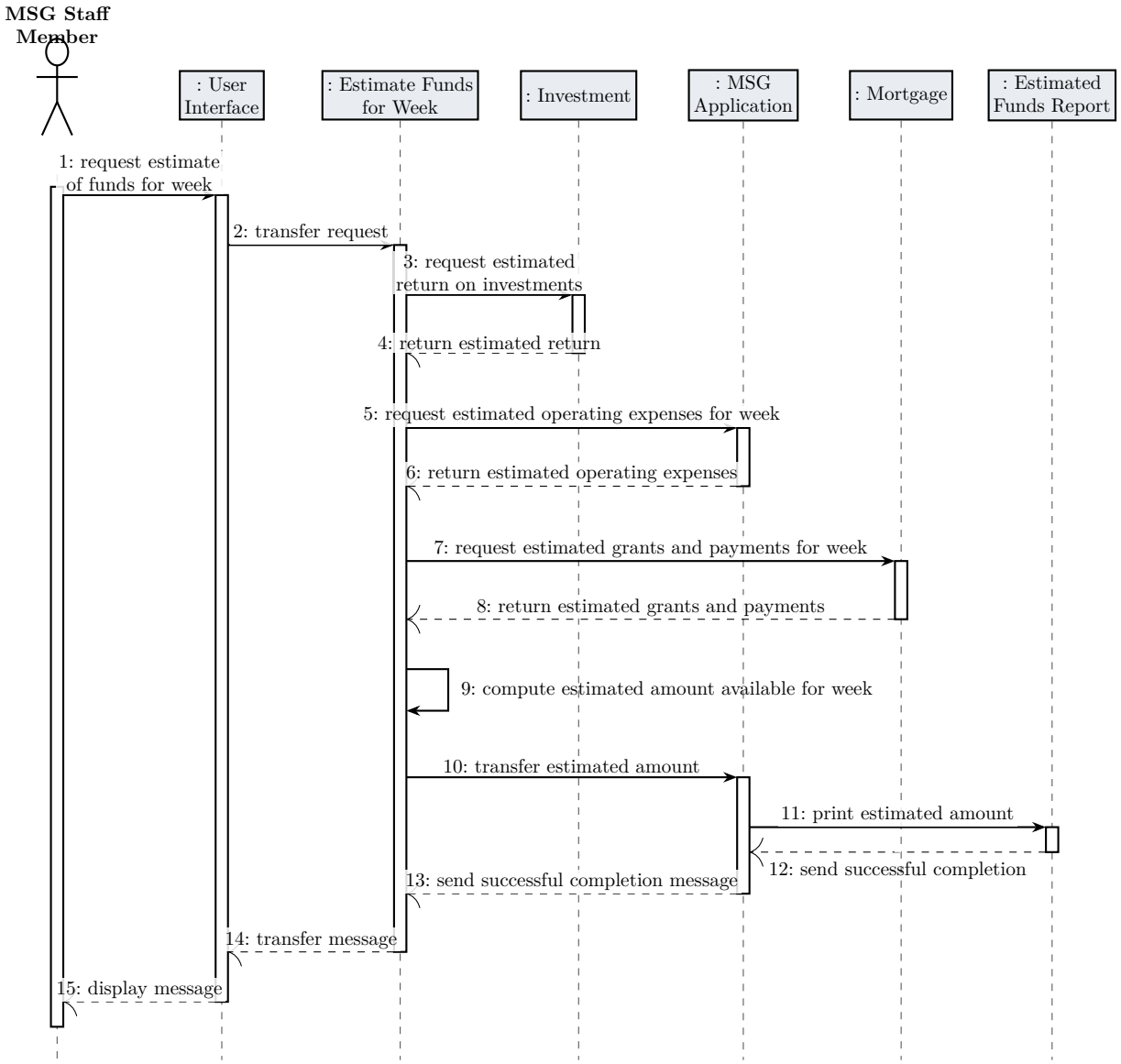
7.6.1 Collaboration diagram



Collaboration diyagramı nesneleri ve aralarındaki **bağlantıları (links)** gösterir; mesajlar bağlantıların üzerinde **numaralı** oklarla verilir. Dikey zaman ekseni yoktur. Mesajların listesi:

#	Gönderen	Alıcı	Mesaj
1	MSG Staff Member	: User Interface Class	Request estimate of funds available for week
2	: User Interface Class	: Estimate Funds for Week Class	Transfer request
3	: Estimate Funds for Week	: Investment Class	Request estimated return on investments for week
4	: Investment Class	: Estimate Funds for Week	Return estimated return on investments for week
5	: Estimate Funds for Week	: MSG Application Class	Request estimated operating expenses for week
6	: MSG Application Class	: Estimate Funds for Week	Return estimated operating expenses for week
7	: Estimate Funds for Week	: Mortgage Class	Request estimated grants and payments for week
8	: Mortgage Class	: Estimate Funds for Week	Return estimated grants and payments for week
9	: Estimate Funds for Week	(kendisi)	Compute estimated amount available for week (**self call**)
10	: Estimate Funds for Week	: MSG Application Class	Transfer estimated amount available for week
11	: MSG Application Class	: Estimated Funds Report Class	Print estimated amount available
12	: Estimated Funds Report	: MSG Application Class	Send successful completion message
13	: MSG Application Class	: Estimate Funds for Week	Send successful completion message
14	: Estimate Funds for Week	: User Interface Class	Transfer successful completion message
15	: User Interface Class	MSG Staff Member	Display successful completion message

7.6.2 Eşdeğer sequence diagram



Sequence diyagramının notasyonu (UML sunumu): aktör ve nesneler soldan sağa dizilir; **zaman yukarıdan aşağı** akar; her nesnenin altında kesikli **hayat çizgisi (lifeline)**; nesne çalışırken ince dikdörtgen **aktivasyon (activation)** kutusu; **çağrılar düz çizgili ok**, **dönüşler kesikli çizgili ok**; nesnenin kendine gönderdiği mesaj (**self call**) kendi hayat çizgisine dönen oktur.

7.6.3 Sequence mı, collaboration mı?

- **Sequence diagram**'ın gücü: mesajların akışını ve **sırasını** açık ve kesin gösterir. Odak **bilgi aktarımı** ise sequence diyagramı üstündür.
- **Collaboration diagram** bir sınıf diyagramına benzer; geliştiriciler **sınıflara** odaklanıyorsa daha kullanışlıdır.
- İkisine birlikte **interaction diagram** denir ve birbirine dönüştürülebilir.

Sequence diyagramı çizerken

- Analiz aşamasında (kitaptaki MSG diyagramları) metotlar henüz belli olmadığı için oklara *"Request estimated return..."* gibi açıklamalar yazılır; hoca: *"Dizayn aşamasına geçip metotlar belli olduktan sonra buralara metotların isimlerini yazmamız gerekecek."* Sınavda (sınıfları sizin tasarladığınız sorularda) metot adlarını yazın.
- Her okun üzerine yazılan çağrı, **okun ucundaki sınıfın bir metodu** olmalıdır (ör. : `Mortgage`'a giden ok `getGrantsAndPayments()`). Kesikli dönüş oklarına dönen değer yazılır.
- Akış use case açıklamasındaki adımlarla birebir uyuşmalı; aktör → boundary → control → entity sırası iyi bir şablondur.
- Koşullu akışlar (if/else) sorulabilir: UML'de `alt` kutusu (combined fragment) ile "[stok var]" / "[stok yok]" bölmeleri çizilir.

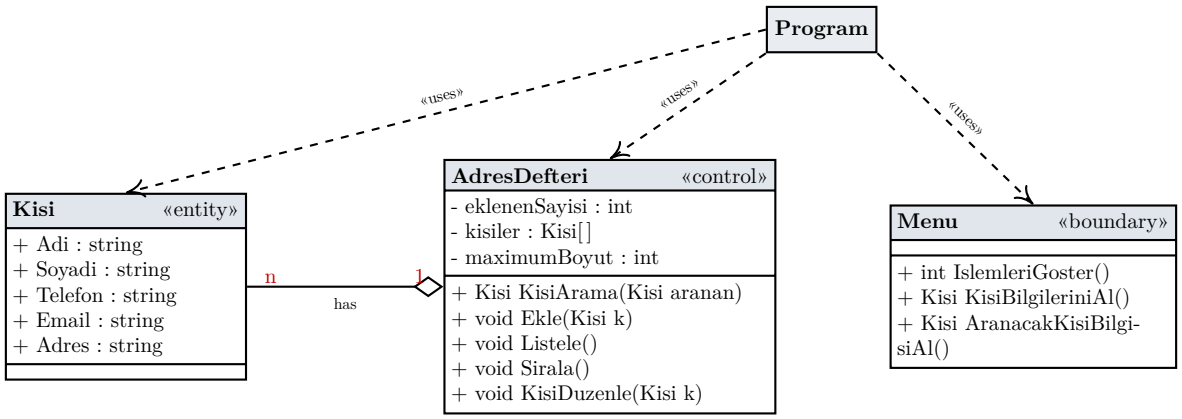
7.7 Hocanın tam çözümlü örneği: Adres Defteri

! Hoca derste vurguladı Kaynak: `Sunular/diagramlar.jpg` ve tahta fotoğrafı

Hocanın kendi el yazısıyla hazırladığı "Bahriye Akay – Nesne Yönelimli Prog. – Adres Defteri Örneği" kâğıdı: use case diyagramı, *Kişi Ekle* açıklaması, sınıf diyagramı (entity/control/boundary), CRC kartları ve *Kişi Ekle* collaboration diyagramı. Notlar: *"tüm usecase'ler için"*, *"tüm işlemler için yapılır"*, *"tüm sınıflar için"*. Yani sınavda senaryo verilirse **bütün** use case'lerin açıklaması, **bütün** sınıfların CRC kartı istenebilir. Use case diyagramı ve açıklamalar için Konu 6, Bölüm 6.6'ya bakın.

Kişi Ekle (hocanın kâğıdındaki hâli): *Kısa tanım:* eklenecek bilgilerin kaydı. *Adım adım tanım:* 1. Menüden ekleme seçilir. 2. Kişi bilgileri alınır. 3. Maksimum boyut aşılmamışsa eklenir. (Use case'ler: Kişi Ekle, Kişi Düzenle, Arama, Kişi Sil, Listele, Sırala, Çıkış; *Kapasite Dolu* ve *İptal* «extends»; Düzenle ve Sil «include» Arama.)

7.7.1 Sınıf diyagramı



- **Kisi** uzun ömürlü bilgiyi tutar ⇒ entity. **Menu** kullanıcıdan bilgi alır, menüyü gösterir ⇒ boundary. **AdresDefteri** kişiler üzerinde ekleme, arama, sıralama işlemlerini yürütür; hoca kâğıtta bunu **control** olarak işaretledi.
- **AdresDefteri ◊— Kisi: aggregation** (*"AdresDefteri has Kişi(ler)"*): bir adres defteri n kişiden oluşan bir topluluk tutar. Elmas AdresDefteri tarafında.
- **Program** diğer sınıfları kullanır (*uses* = dependency).

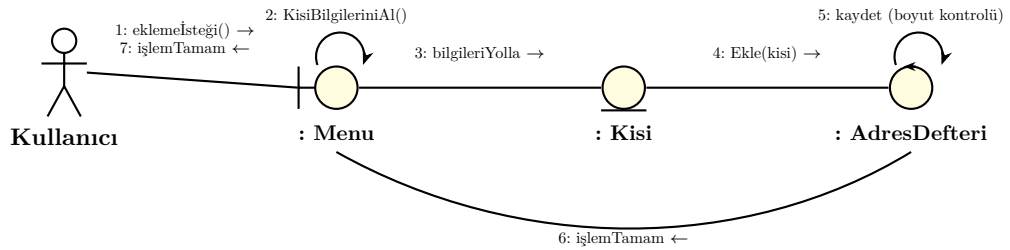
- Hocanın tahtadaki (daha yeni) sürümünde ise kontrol ayrı bir **DefterController** sınıfına alınmış, AdresDefteri kişileri tutan sınıf olarak kalmış, kullanıcı arayüzü **KullaniciArayuzu** (AnaMenuUret, EkleMenu, SilMenu …) ve **Raporlama** (printRapor) boundary sınıfları eklenmiş; Kisi ile AdresDefteri arası **dolu elmas (composition)** çizilmiş. İki çözüm de geçerlidir: ilişki türünü **gerekçesiyle** yazın (kişiler defterle birlikte silinir ⇒ composition; kişi defterden bağımsız yaşayabilir ⇒ aggregation).

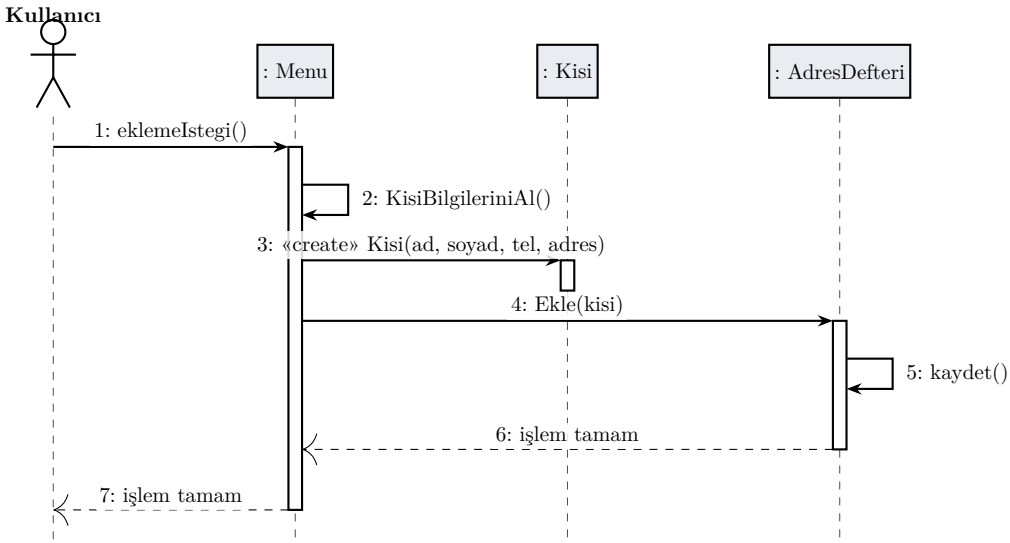
7.7.2 CRC kartları (hocanın kâğıdından)

CLASS: AdresDefteri	**CLASS: Menu**	**CLASS: Program**
Responsibility: kişi ekler; kişi siler; kişi düzenler; arama yapar; sıralama yapar	**Responsibility:** ana menüyü gösterir; kişi ekleme bilgilerini alır; kişi arama bilgisini alır	**Responsibility:** menüyü listelemek için Menu nesnesine mesaj yollar; işlemler için AdresDefteri'ne mesaj yollar
Collaboration: Kisi, Program	**Collaboration:** Kisi, AdresDefteri, Program	**Collaboration:** Kisi, Menu, AdresDefteri

Kisi için de: *Responsibility:* kişi bilgilerini tutar ve verir; *Collaboration:* AdresDefteri.

7.7.3 Kişi Ekle: collaboration ve sequence diyagramı





Akış: kullanıcı ekleme ister (1), menü kişi bilgilerini alır (2, self call), bilgilerle Kisi nesnesi oluşturulur (3), kişi adres defterine eklenmek üzere gönderilir (4), adres defteri maksimum boyutu kontrol edip kaydeder (5, self call), sonuç geriye bildirilir (6, 7). Kapasite doluysa *Kapasite Dolu* extend akışı çalışır (`alt` çerçevesiyle gösterilebilir).

7.8 Diğer konular

- **Specification document in the Unified Process:** Unified Process **use-case driven**'dır; use case'ler ve onlardan türeyen artifact'lar geleneksel metin spesifikasyonunun yerini alır ve aynı sözleşme rolünü oynar. Müşteri ürünün nasıl çalışacağını use case + senaryolardan da rapid prototype'tan da anlayabilir; fark: use case'ler her seferinde inceltilir, rapid prototype atılır. Yine de kullanıcı arayüzünün prototipi (örnek ekranlar, raporlar) gerekir.
- **Aktörleri ve use case'leri bulmak:** bir kişinin ürünle etkileşebileceği her rolü düşün; aktörler bireyler değil **rollerdir**. Unified Process rol için *worker* terimini kullanır (Applicants ve Borrowers iki ayrı worker). İş modelinin yazılıma karşılık gelen alt kümesindeki aktörler, aradığımız aktörlerdir; her rol için bir veya daha fazla use case vardır.
- **OOA'nın zorlukları:** nesne yönelimli tasarımın sınırını geçme; **sınıflara henüz metot atama** (stepwise refinement sırasında metotları yeniden dağıtmak boşa emektir).
- **Metrikler:** size, cost, duration, effort, quality; OOA'nın boyut ölçüsü: UML diyagramlarının sayfa sayısı.

7.9 Kısa tanım soruları

Entity / boundary / control class Models long-lived information / models the interaction between the product and its environment (input, output, reports) / models complex computations and algorithms.

Scenario An instance of a use case.

Noun extraction Describe the product in a single paragraph, identify the nouns, discard those outside the problem boundary and the abstract nouns; the rest are candidate classes.

CRC card Class–Responsibility–Collaboration card: the name of the class, its responsibilities and the classes it invokes.

God class A class that is exposed to too much information and has too much control (an antipattern).

Use-case realization The process of extending and refining use cases; the realization of a scenario is depicted by a sequence or collaboration diagram.

Collaboration diagram Shows the objects, the links between them and the messages numbered in the order in which they are sent.

Sequence diagram Shows the objects and the messages between them in time order (top to bottom); calls are solid arrows, returns dashed arrows.

8 UML: Sınıf İlişkileri, Sequence ve Diğer Diyagramlar

! Hoca derste vurguladı Hafta 9 ve 2025 vizesi

Bu konu kitabın değil, hocanın paylaştığı `1uml.pdf` sunumunun ("Introduction to UML") konusudur. Hafta 9 sonunda: *"Sınavımız bugünkü konulara kadar olan kısımları kapsıyor"* ⇒ vize kapsamındadır. *"Geçmiş yıllarda ben bununla ilgili sorular soruyordum sınavda: klas diagramını vererek hem sınıfların arasındaki ilişkileri yorumlamanızı hem de sayıları yorumlamanızı istiyordum. Yani sadece bu bir binary association falan diyerek bırakmamalıyız."* 2025 WhatsApp: *"uml pdf'inin 35. sayfasında bir class example var; böyle bir class diyagramı verip aralarındaki ilişkileri yorumlamanızı isteyebilirim ama detaylı isterim."* 2025–26 vizesinde: *rent a car otomasyonu, okların çeşitleri (aggregation, composition, inheritance vs.) ve işlevleri* + aynı otomasyonun *sequence diyagramı* (24 puan).

8.1 UML nedir, neden kullanılır?

- **Unified Modeling Language**: OMG (Object Management Group) standardı; Booch, Rumbaugh ve Jacobson'ın çalışmalarına dayanır (1994 Booch–Jacobson, 1995 Rumbaugh katıldı, 1997 UML 1.1, bugün UML 2.x).
- Belgeleri ve yazılımı ifade etmek ve tasarlamak için bir **modelleme dilidir**; özellikle nesne yönelimli tasarımda yararlıdır. **Bir süreç değildir** ve **programlama dilinden bağımsızdır**.
- Neden? Açık standart, yazılım sistemlerini *belirlemek, görselleştirmek, inşa etmek ve belgelemek* için grafik notasyon; ilk tasarımdan ayrıntılı tasarıma kadar bütün yaşam döngüsünde kullanılır; müşteri ve geliştiriciler arasında anlamayı ve iletişimi artırır; birçok araç destekler.
- **Model** bir sistemin bir alt kümesini tanımlayan soyutlamadır; **view** bir modelin seçilmiş yönlerini gösterir; **notation** görünümleri çizmek için grafik/metin kurallarıdır. (Örnek: sistem = uçak; modeller = uçuş simülatörü, ölçekli model; görünümler = planlar, elektrik tesisatı, yakıt sistemi.)
- UML çok diyagramlıdır; her diyagram bir modele bakan bir görünümdür. *Problemlerin %80'i UML'in %20'siyle modellenebilir.*

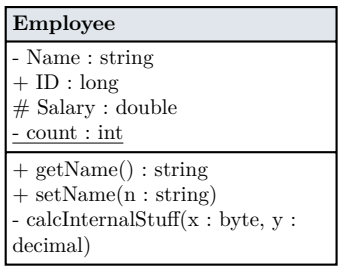
UML baseline diyagramları: use case diagrams, class diagrams, package diagrams, interaction diagrams (sequence, collaboration), activity diagrams, state transition diagrams, (component ve) deployment diagrams.

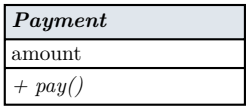
Statik ve dinamik: *Statik tasarım* kod yapısını ve nesne ilişkilerini anlatır, değişmez (class, object, package, component, deployment diyagramları). *Dinamik tasarım* nesneler arası iletişimi, olay sıralarını gösterir, senaryoya göre değişir (sequence, collaboration, activity, state diyagramları).

8.2 Class diagram

Sistemi, sınıflarını ve aralarındaki ilişkileri göstererek özetler. Class diyagramları **statiktir**: neyin neyle etkileştiğini gösterir ama etkileşince *ne olduğunu* göstermez. Her sınıfın nitelik ve işlemlerini de gösterir; sistemin genel mimarisini anlatmanın iyi bir yoludur. Üç bakış açısıyla çizilir: **conceptual** (yazılımdan ve dilden bağımsız), **specification** (yazılımın arayüzlerine odaklı), **implementation** (gerçekleştirmeye odaklı).

8.2.1 Sınıf notasyonu



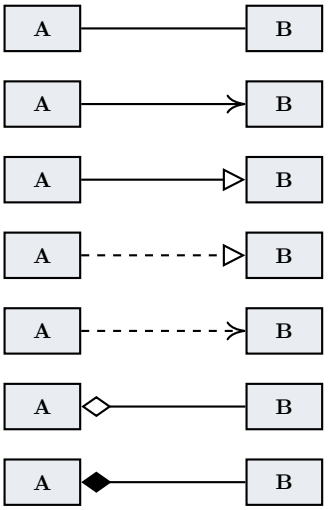


italik ad = **abstract class**

1. bölme: sınıf adı (**tek zorunlu bilgi**)
2. bölme: nitelikler (attributes) ve tipleri
3. bölme: işlemler (operations) ve imzaları

Görünürlük: `-` private, `+` public, `#` protected; **altı çizili** = static (sınıfın bütün nesnelerinde ortak). **Instance** (nesne) gösterimi: adı altı çizilidir ve sınıfını içerebilir (tarif_1974 : TariffSchedule); nitelikler değerleriyle yazılır.

8.2.2 Sınıflar arası ilişkiler



Association
düz çizgi: iki sınıf birbirinden haberdar

Directed (unary) association
açık uçlu ok: A, B'yi bilir; B, A'yı bilmez

Inheritance (generalization)
boş üçgen ata sınıfa doğru: "is-a"

Realization
kesikli çizgi + boş üçgen arayüze doğru: "can-do"

Dependency
kesikli açık ok: "uses-a" (geçici kullanım)

Aggregation
boş elmas **bütün (topluluk)** tarafında: zayıf "has-a"

Composition
dolu elmas **bütün** tarafında: güçlü "has-a", yaşam döngüsü kontrolü

İlişki	Anlamı	Örnek
Association (binary)	Bir sınıfın işini yapabilmek için diğerini bilmesi; mesajlaşma (metotlarına çağrı). İki sınıf da birbirinden haberdar.	Öğrenci – Öğretmen; Customer – Order
Unary / directed association	Tek yönlü haberleşme: A, B'nin metodunu çağırır, B'nin A'dan haberi yok. **Ok çağrılan sınıfa doğrudur.**	OrderDetail → Item
Reflexive association	Bir sınıfın kendisiyle ilişkisi (sınıfın birden çok rolü varsa).	Employee (yönetici) – Employee (çalışan)
Aggregation	"Topluluk–üye" ilişkisi; bir sınıf diğerinin nesnelerinden oluşan bir topluluğu içerir. **Tek sahiplik yok**: parça bütün olmadan yaşayabilir.	Takım ◇– Oyuncu (oyuncu takımsız yaşar); Fakülte ◇– Öğretim üyesi
Composition	Aggregation'ın güçlü hâli: **bütün, parçaların oluşturulmasından ve silinmesinden sorumludur** (lifetime control); parça yalnızca bir bütüne ait olabilir ve bütün olmadan var olamaz.	Ev ◆– Oda; Team ◆– Employee (`new Employee()` / `delete` Team içinde)
Inheritance / generalization	Ata sınıftaki öğelerin alt sınıflarca paylaşılması; "is-a". Ayrıca alt sınıfların belirli özelliklere sahip olduğunun garantisidir.	Gül → Çiçek; Credit, Cash, Check → Payment
Realization	Bir sınıfın bir arayüzü (interface) gerçekleştirmesi; arayüz sadece metot adlarını ve parametrelerini içerir.	Kuş - - ▷ «interface» Uçabilir
Dependency	Bir sınıfın diğerini geçici olarak kullanması (parametre, yerel değişken); diğerindeki değişiklik onu etkileyebilir.	Rapor - - > Yazıcı; Librarian - - «use» > Search

Elmasın yeri

"İçi boş elmas işareti topluluğun olduğu tarafa çizilir, ona bitişik çizilir. Bazı arkadaşlarımız sınavlarda getirip ortaya çiziyorlar veya ters tarafa çizebiliyorlar. Bunlar yanlış." Elmas her zaman **bütünün (collection/whole)** yanındadır. Kalıtım üçgeni her zaman **ata sınıfa** bakar. Directed association oku **çağrılan** sınıfa bakar.

8.2.3 İlişkilerin C# karşılıkları

```
class A { B myB; public void DoSomething() { myB.Service(); } }   // binary association:
class B { A myA; public void Operation()   { myA.DoSomething(); } } //   ikisi de birbirini
    cagirir

class C { B myB; public void Do() { myB.Service(); } }   // unary association: C -> B

class Takim { List<Oyuncu> oyuncular;                    // aggregation:
              public void Ekle(Oyuncu o) { oyuncular.Add(o); } } // oyuncu disarida yaratilir

class Ev { List<Oda> odalar = new List<Oda>();           // composition:
           public Ev() { odalar.Add(new Oda()); } }      // parcayi butun yaratir ve yok eder
```

```
class Gul : Cicek { }                                    // inheritance (is-a)
interface IUcabilir { void Uc(); }
class Kus : IUcabilir { public void Uc() { } }           // realization (can-do)
class Rapor { public void Bas(Yazici y) { y.Yaz(); } }   // dependency (uses-a)
```

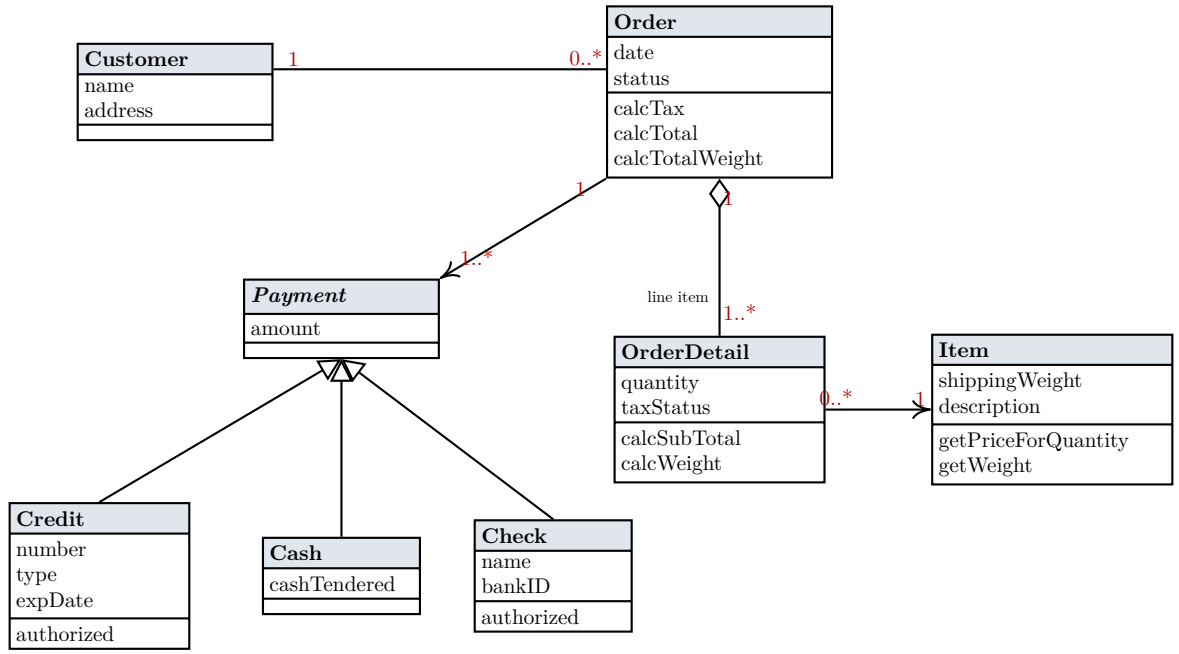
8.2.4 Çokluklar (multiplicities)

İlişki çizgilerinin uçlarına yazılan sayılar, karşı sınıftan **kaç nesne** ile ilişki kurulduğunu gösterir:

Gösterim	Anlamı
`1`	tam olarak bir
`0..1`	sıfır ya da bir
`0..*` ya da `*`	sıfır veya daha fazla (sınırsız, hiç olmayabilir)
`1..*`	en az bir
`n..m`	n ile m arası

İlişki uçlarına **rol adları** da yazılabilir (ör. Team tarafında *group*, Employee tarafında *individual*; Order–OrderDetail'da *line item*). Okuma kuralı: "**Bir** A'nın, **B tarafındaki sayı kadar** B'si vardır."

8.3 UML sunumu sayfa 35: sınıf diyagramını yorumlama



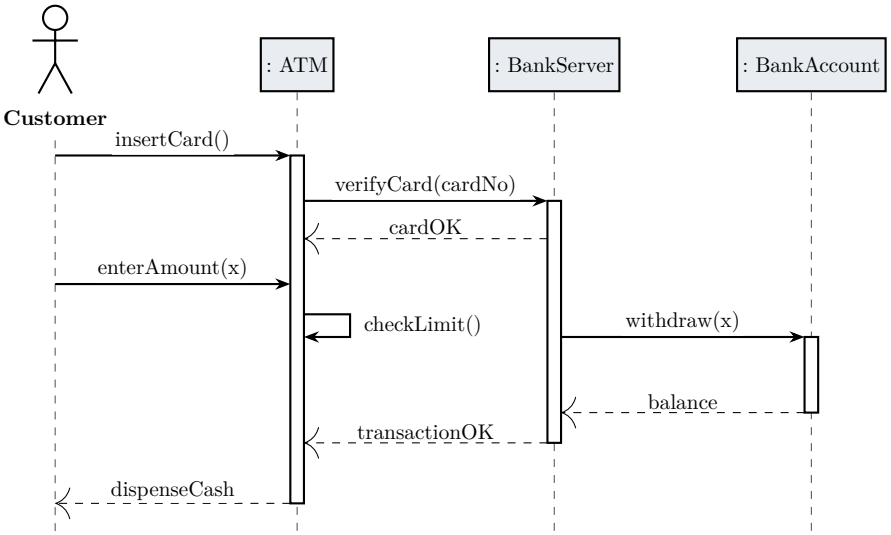
Hocanın derste yaptığı yorum (sınavda beklenen ayrıntı düzeyi)

Önce sınıfları tanıt, sonra ilişkileri, sonra sayıları yorumla. Bu bir sipariş ve ödeme sistemine ait sınıf diyagramıdır.

1. **Customer – Order: binary association** (düz çizgi). Customer ve Order birbirinden haberdardır, birbirlerinin metotlarını çağırabilir. Çokluk: **bir müşterinin sıfır veya daha fazla siparişi** olabilir; her sipariş **tek bir müşteriye** aittir.
2. **Order ◊– OrderDetail: aggregation** (boş elmas Order tarafında). Bir sipariş, sipariş detaylarından (*line item*) oluşan bir topluluktur: **bir siparişin bir veya daha fazla sipariş detayı** vardır ("5 hamburger, 3 salata"). Her detay bir siparişe aittir.

3. **OrderDetail → Item: unary (directed) association**. OrderDetail, Item'ın metotlarını (`getPriceForQuantity`, `getWeight`) çağırır; Item'ın OrderDetail'den haberi yoktur. Çokluk: her sipariş detayı **tam bir** ürüne karşılık gelir; bir ürün **sıfır veya daha fazla** sipariş detayında geçebilir. (Hoca burada "bence 1..* olmalıydı" yorumunu da yaptı: sipariş verilmeden detay olmaz.)
4. **Order → Payment: unary association**. Bir siparişin **bir veya daha fazla ödemesi** olabilir (bir kısmı kart, bir kısmı nakit); her ödeme bir siparişe aittir.
5. **Credit, Cash, Check → Payment: generalization (inheritance)**. Üçü de Payment'ın özel hâlleridir; hepsinde ortak `amount` vardır, ayrıca kendilerine özgü nitelikleri (Credit: number, type, expDate; Check: name, bankID) ve işlevleri (`authorized`) vardır.
6. **Payment adının italik olması**: **abstract class**; gövdesi ata sınıfta yazılmamış üyeler alt sınıflarda tamamlanacaktır; Payment'tan doğrudan nesne üretilmez.

8.4 Sequence diagram notasyonu



- Aktör (use case'ten gelen) ve nesneler (`nesneAdı : SınıfAdı` veya `: SınıfAdı`) en üstte soldan sağa dizilir.
- **Zaman yukarıdan aşağı** akar; nesnelerin altında kesikli **lifeline**, çalıştıkları süre boyunca ince dikdörtgen **activation**.
- **Calls = düz çizgili oklar** (mesaj = çağrılan sınıfın metodu), **returns = kesikli oklar**.
- Nesnenin kendine mesajı (**self call**); nesnenin oluşturulması (`new`) ve yok edilmesi (lifeline sonunda **X**).
- Eğik çizgiler mesaj gecikmesini gösterir (gerçek zamanlı sistemler); mesajlar kesişirse genellikle sorun vardır (*race condition*).
- UML 2'de koşul ve döngüler **combined fragment** ile çizilir: `alt` (if/else), `opt` (if), `loop`.

UML sunumundaki örnekler: *Alarm System* (alarm çalınca zil çalar, ekrana mesaj yazılır, izleme servisine haber verilir) ve *Hotel Reservation*.

8.5 Diğer UML diyagramları

Diyagram	Özeti
Collaboration (communication)	Sequence ile benzer bilgi, ama **dikey zaman ekseni yok**; yerine nesneler arası **bağlantılar** (düz çizgi) ve bağlantılar üzerinde yönlü, numaralı mesajlar. Vurgu statik bağlantılardadır.
Object diagram	Class diyagramının örnekleri ve aralarındaki bağlantılar; belirli bir andaki nesnelerin "fotoğrafı". Biçim: `örnekAdı : SınıfAdı` + nitelik değerleri.
Package diagram	Karmaşık class diyagramlarını mantıksal paketlere gruplar. Paket: üstünde küçük sekmesi olan dikdörtgen; kesikli oklar **bağımlılıktır** (diğer paketteki değişiklik bu paketi değiştirmeye zorlayabilir).
Activity diagram	"Süslü akış diyagramı": tek bir sürecin etkinlik akışı. Köşeleri yuvarlatılmış kutular (etkinlik), **swim lanes** (hangi nesne hangi etkinlikten sorumlu), **baklava** (dallanma), **kalın çubuk** (fork/join: paralel etkinlikler), başlangıç ve bitiş daireleri.
State transition (statechart)	"Süslü DFA": bir nesnenin olası durumları ve durum değişimine yol açan geçişler. Durum = yuvarlatılmış dikdörtgen; geçiş oku üzerinde `Event [Condition] / Action`. Durumun içinde `do/ activity`, `entry/ action`, `exit/ action`. Yerel değişken tutabilir (kütüphane örneği: `Borrow / N = N+1`).
Component diagram	Sistemdeki bileşenler, arayüzleri ve bağımlılıkları; sistemin fiziksel yapısı. Bileşen: sol üstünde iki sekme olan dikdörtgen; arayüz: daire + çizgi (lolipop); bağımlılık: kesikli ok.
Deployment diagram	Kurulmuş sistemin donanım ve yazılım fiziksel mimarisi: **node**'lar (makine/cihaz) ve içlerindeki bileşenler; hangi paket hangi sunucuda (hocanın emlak–banka örneği).

8.6 Kısa tanım soruları

UML A standard graphical modeling language (OMG) for specifying, visualizing, constructing and documenting software; not a process and independent of the implementation language.

Association A relationship between instances of two classes, where one class must know about the other to do its work (solid line; arrow if one-directional).

Aggregation An association where one class belongs to a collection (collection–member); hollow diamond on the collection side; no sole ownership.

Composition A strong form of aggregation with lifetime control: the parts cannot exist without the whole; filled diamond on the whole side.

Inheritance An "is-a" relationship; a hollow triangle pointing to the superclass.

Realization A class implements an interface; dashed line with a hollow triangle pointing to the interface.

Dependency A "uses-a" relationship; a change in one class may affect the other; dashed open arrow.

Multiplicity The number of instances of one class that can be associated with one instance of the other (1, 0..1, *, 1..*, n..m).